



Review and exercises: Hardware components

Faculty: Computer science – IT1

Instructor: Viet Hoang Pham

Email: vietph@ptit.edu.vn

Phone: 0945296388

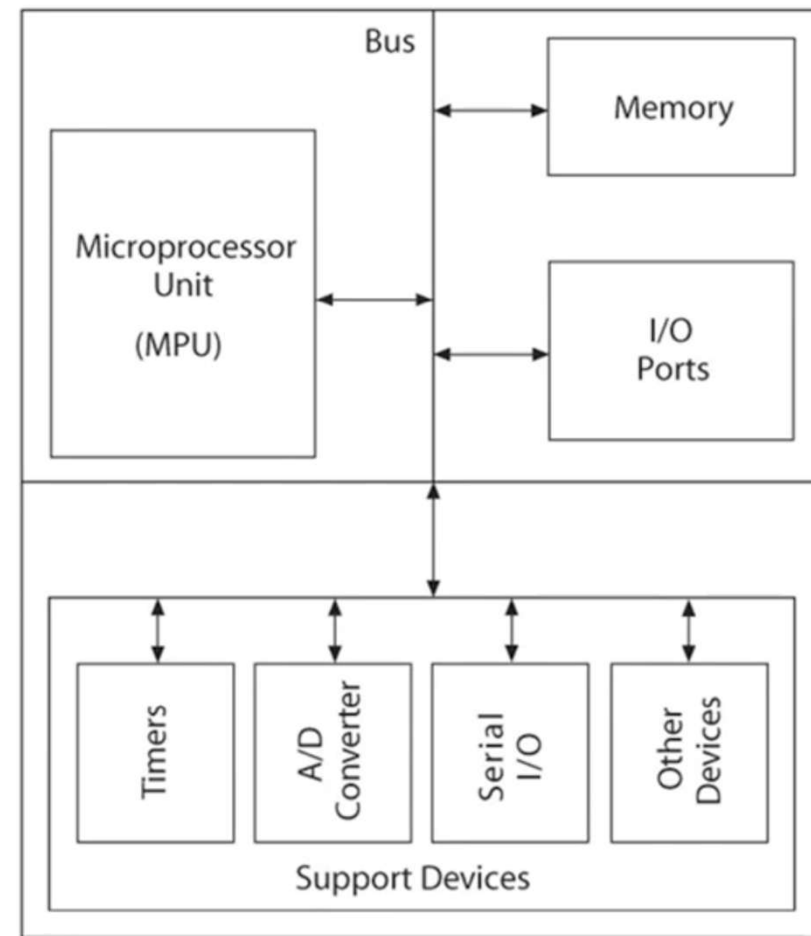
- Central processing unit - CPU
- Memory and memory design
- Pairing with peripheral devices
- 8051 microprocessor: Architecture, memory and practical interface

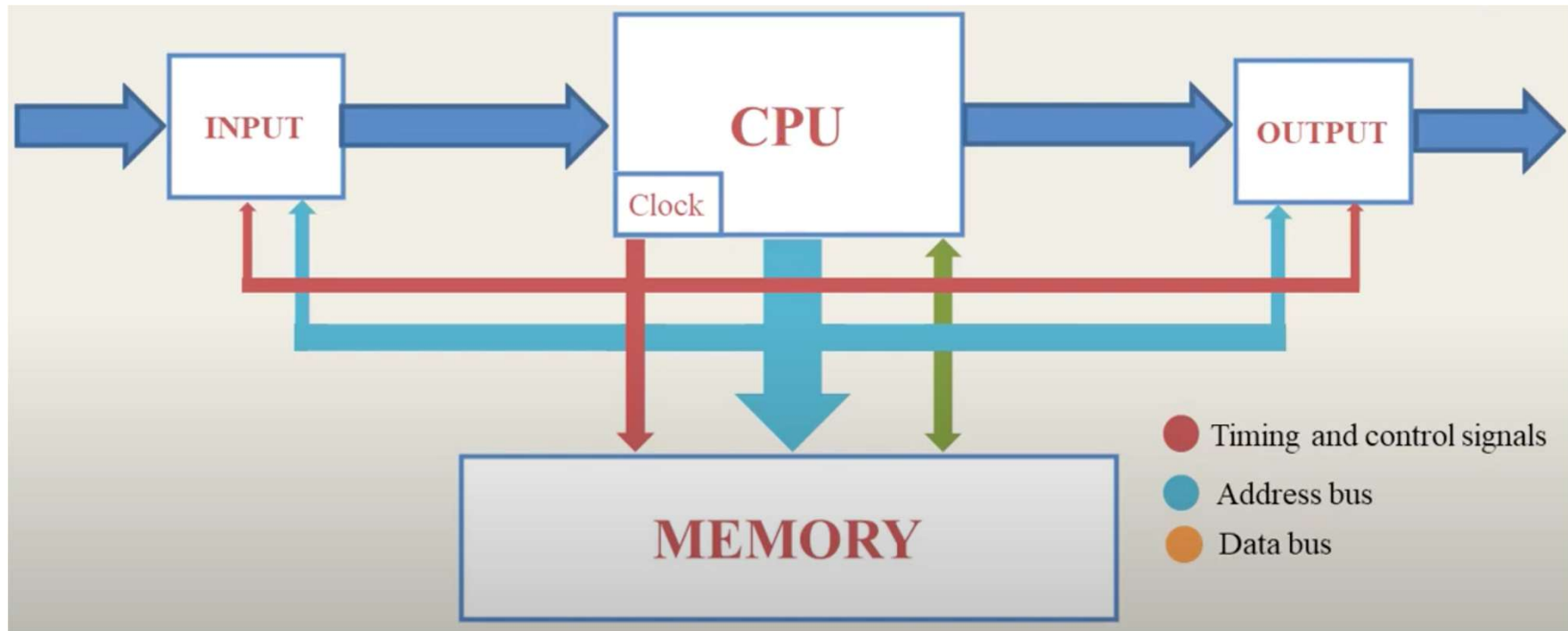
❖ Microprocessor

- Characteristics
 - Contains only a CPU
 - Requires external RAM, ROM, and I/O
 - High performance, runs an operating system (OS)
- Used in:
 - PC
 - Server
 - Heavy computing systems
- Examples: Intel x86, AMD Ryzen

❖ Microcontroller

- Built-in:
 - CPU, RAM, Flash,
 - Timer, ADC, GPIO,
 - UART/SPI/I2C
- Used in:
 - IoT,
 - Robot,
 - Household appliances,
 - Industrial control systems
- Examples: STMicroelectronics STM32, Microchip Technology PIC, Espressif Systems ESP32



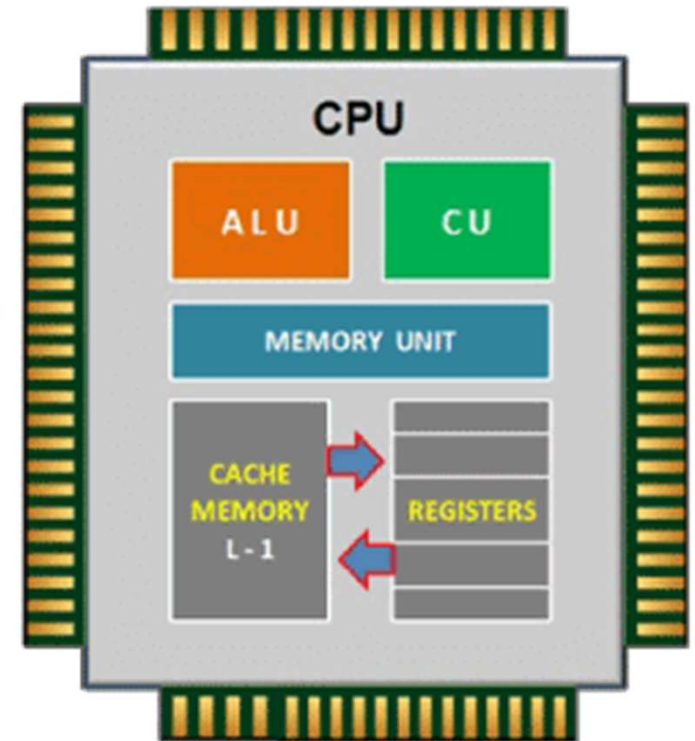


- CPU (Central Processing Unit): Executes instructions and processes data
- Memory: Program and data memory
- Peripheral: Input / output devices
- Bus: transmits signals between components

❖ Basic CPU block diagram includes:

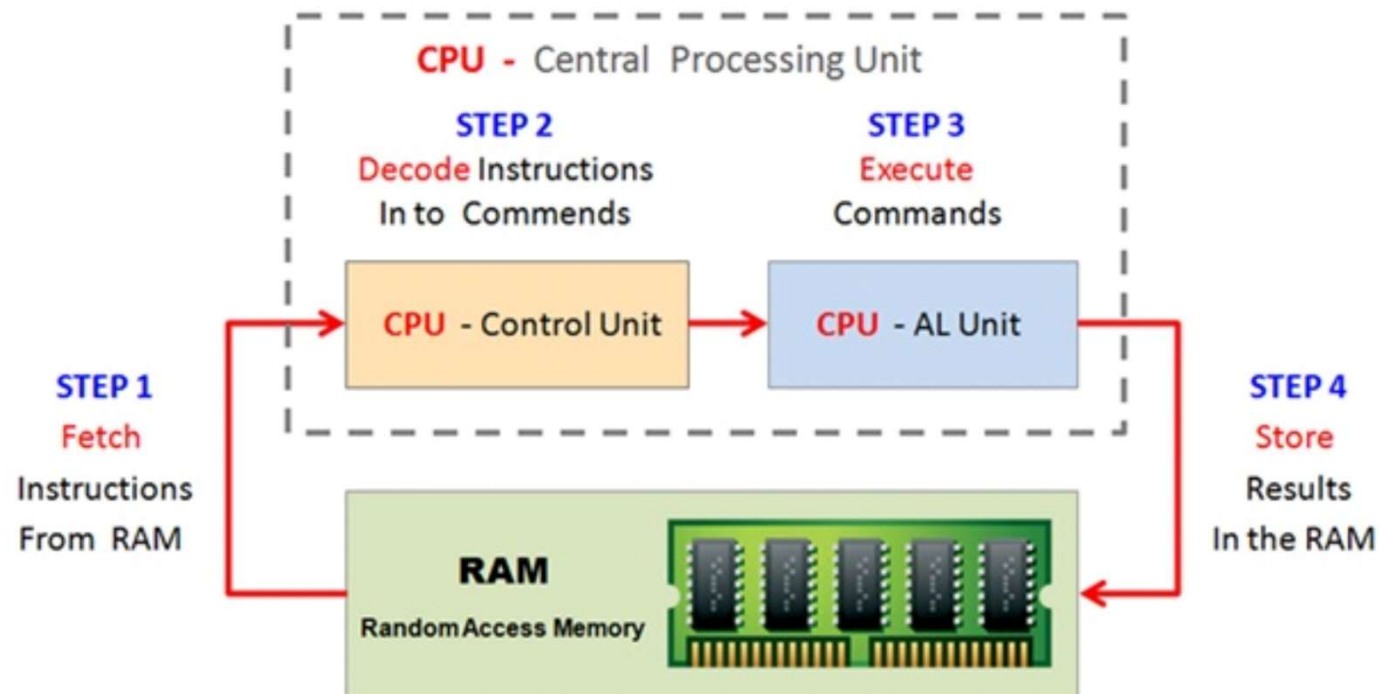
- ALU: Computational and logic unit
- CU: Control unit
- Memory Unit: Some memory is built into the CPU to support storage and fast calculations. Includes registers and cache.

Central Processing Unit



❖ Process of executing a command:

1. Load instructions from memory into CU
2. CU Decodes the command
3. The ALU executes the command
4. The results are saved in memory



❖ Instruction Set Architecture (ISA):

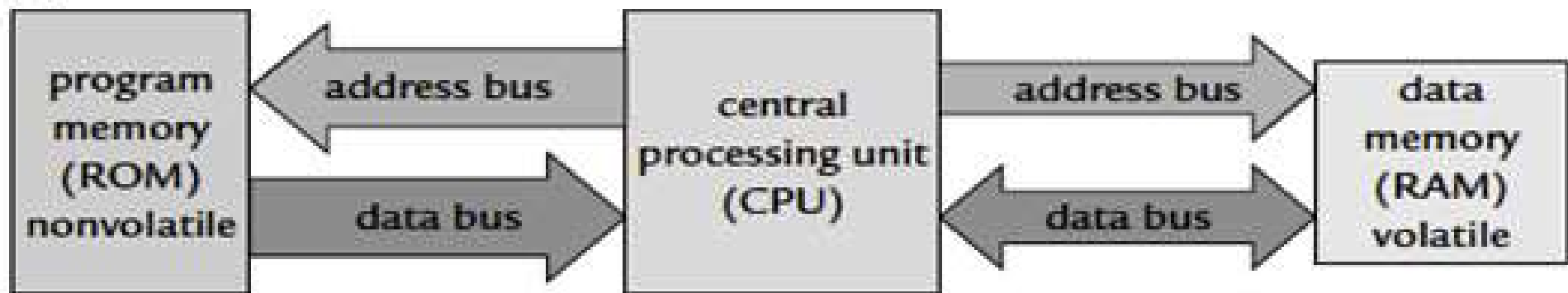
- The instruction set architecture defines how the CPU interacts with data and memory through:
 - Addressing Modes
 - Data Storage Organization
 - Register Sets
- In embedded system design, the choice of instruction set architecture directly affects:
 - Processing Performance
 - Source Code Optimization
 - Development Tools
- There are two types of instruction set architectures :
 - CISC (Complex Instruction Set Computing)
 - RISC (Reduce Instruction Set Computing)

❖ Kiến trúc tập lệnh ISA (Instruction Set Architecture):

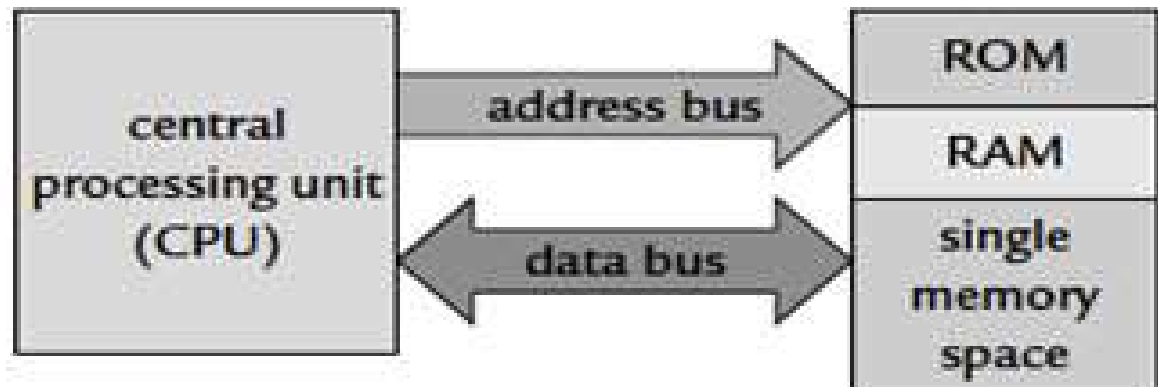
Đặc điểm	CISC (Tập lệnh phức hợp)	RISC (Tập lệnh rút gọn)
Truy cập bộ nhớ	Bất kỳ lệnh nào cũng có thể truy cập	Chỉ lệnh Load và Store
Độ dài lệnh	Thay đổi tùy theo độ phức tạp	Cố định
Thời gian thực hiện	Nhiều nhịp chu kỳ	Thường chỉ một nhịp chu kỳ
Thanh ghi	Chỉ có một tập thanh ghi	Có nhiều tập thanh ghi
Cơ chế thực thi	Thông dịch qua vi mã (microcode)	Phản cứng thực thi trực tiếp
Hỗ trợ Pipeline	Không hỗ trợ (vẫn có thể)	Hỗ trợ mạnh mẽ
Kích thước mã	Nhỏ gọn, tối ưu bộ nhớ	Lớn hơn

❖ Architectures

(a) Harvard architecture



(b) von Neumann architecture

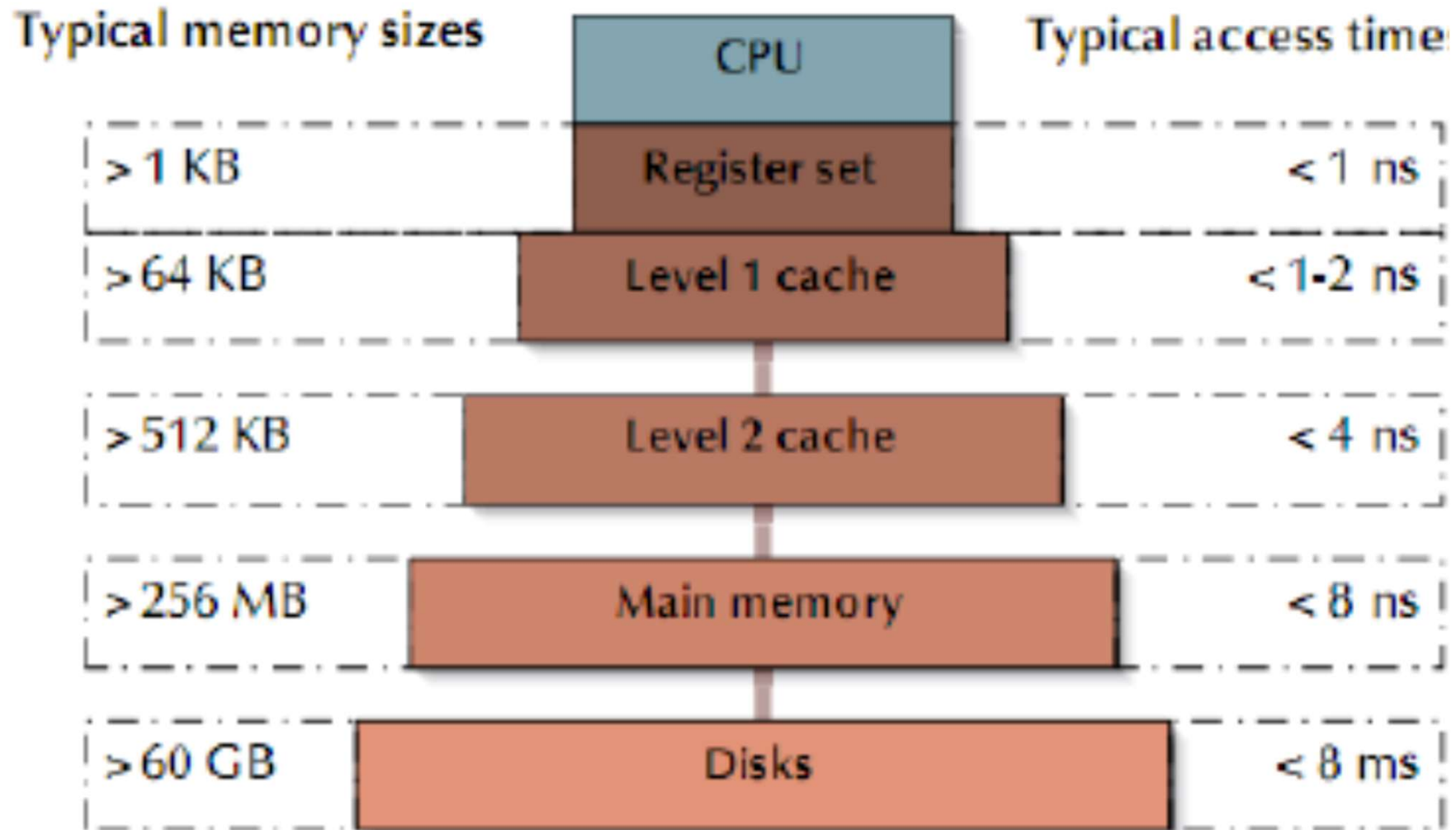


❖ Memory is a medium for storing information, including programs and data.

Some basic information about computer memory:

- Capacity is determined by the formula: $\text{Capacity} = 2^n \times m$. Where: n is the number of address bits, i.e., the number of address lines that the CPU uses to select a memory cell, and m is the length of each memory cell.
- Memory access time. Access time, denoted by t_a , is the time from when the CPU places an address on the BUS until the data actually appears on the data BUS.
- Read cycle (t_{RC}) and write cycle (t_{WC}) indicate the shortest time interval between two consecutive accesses to the same memory cell.
- Wait state, denoted by t_w : are the clock cycles during which the CPU is forced to pause, waiting for the memory to return data.

❖ Memory hierarchy model:



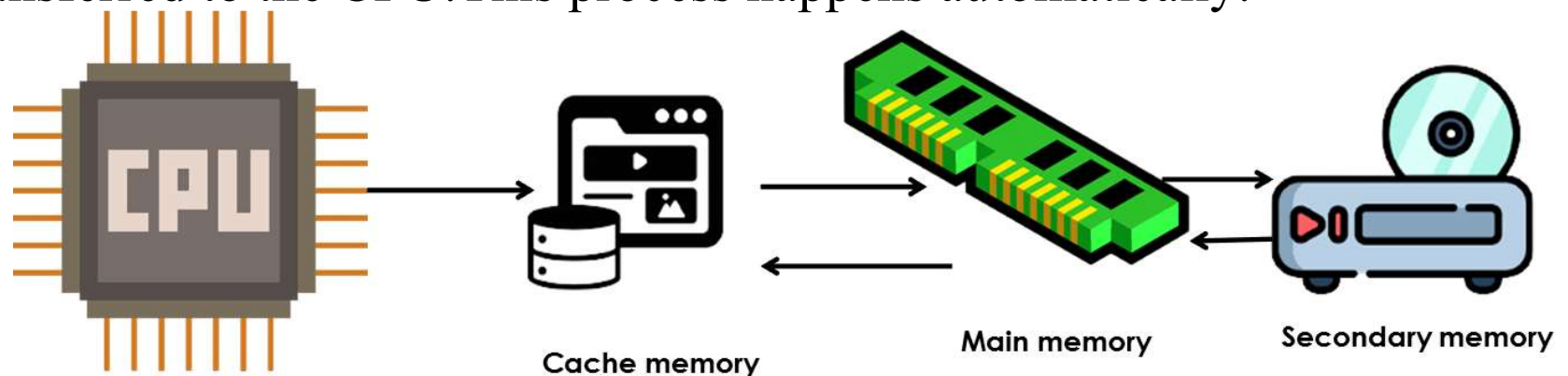
❖ **Memory map** (bản đồ bộ nhớ) là cách **phân bố không gian địa chỉ** để CPU có thể truy cập: bộ nhớ chương trình, bộ nhớ dữ liệu, thanh ghi ngoại vi, thiết bị I/O

- CPU chỉ làm việc với: địa chỉ (address) và dữ liệu (data)
- Nếu không có memory map, CPU không thể giao tiếp phần cứng.
- CPU có N bit address bus $\rightarrow$ Không gian địa chỉ $= 2^N$
- Ví dụ đơn giản cho CPU 16-bit
 - 0000–7FFF Flash
 - 8000–9FFF RAM
 - A000–A0FF Peripheral
 - A100–FFFF Free
- Memory map cho phép truy cập ngoại vi như biến, không cần lệnh I/O riêng, và cho phép thêm thiết bị bằng ánh xạ địa chỉ.

❖ Registers:

- Registers are ultra-fast memory located within the CPU.
- Used for: Storing operands, storing addresses, controlling CPU operations.
- Important register types:
 - Special registers: accumulator registers, etc.
 - General-purpose registers: Store temporary data.
 - Program Counter (PC): Stores the address of the next instruction.
 - Instruction Register (IR): Contains OpCode.
 - Stack Pointer (SP): Manages the stack.
 - Status Register: Zero, Carry, and Overflow flags.
 - Control Registers: Controls peripherals.

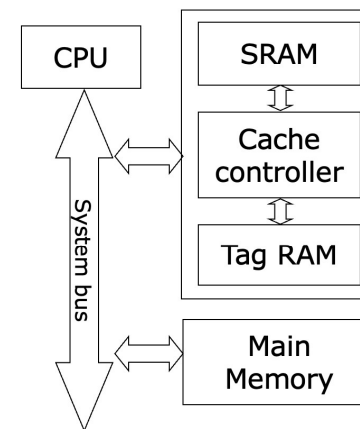
- ❖ Cache is a memory component in a computer's memory hierarchy. It acts as an intermediary, transferring data from main memory to the CPU and vice versa.
- ❖ The cache acts as a data intermediary:
 - When the CPU needs data or a command, the system prioritizes searching the cache first.
 - If the data is already in the cache, the CPU can access it very quickly, without waiting for RAM.
 - If the data is not in the cache, it will be retrieved from main memory, placed in the cache, and
 - then transferred to the CPU. This process happens automatically.



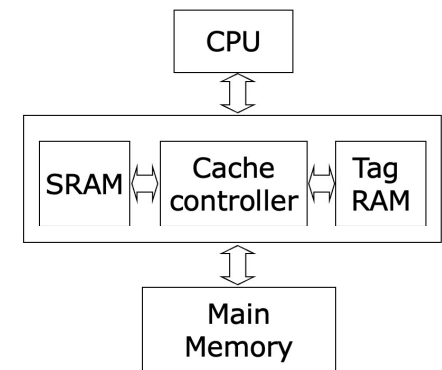
- ❖ Cache Hit: A hit is the event where the CPU accesses data/instructions found in the cache.
 - The probability of a hit occurring is called the hit coefficient H : $0 \leq H \leq 1$
- ❖ Cache Miss: A miss is the event where the CPU accesses data/instructions not found in the cache.
 - The likelihood of a miss occurring is called the miss coefficient or $1-H$
- ❖ There are two main cache layout architectures
 - Look-aside architecture (peer-to-peer): The CPU, cache, and main memory are all connected in parallel on the system bus.

When the CPU issues a memory access request, the cache and main memory simultaneously check the requested address.

 - Look-through architecture (hierarchical): The CPU only works directly with the cache; the cache acts as an intermediary between the CPU and main memory.



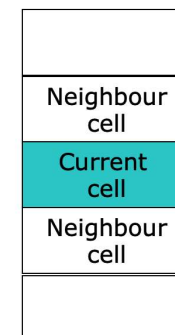
Look aside



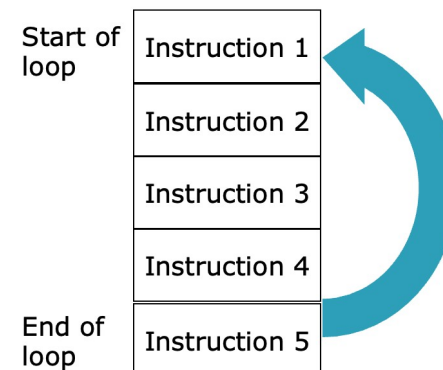
Look through

- ❖ Cache memory is considered intelligent memory because it not only passively stores data, but also actively assists the CPU in processing.
- ❖ Cache has the ability to anticipate the CPU's need for instructions and data.

- Spatial locality principle: When the CPU accesses a memory location, it is highly likely that the CPU will soon access neighboring memory locations.



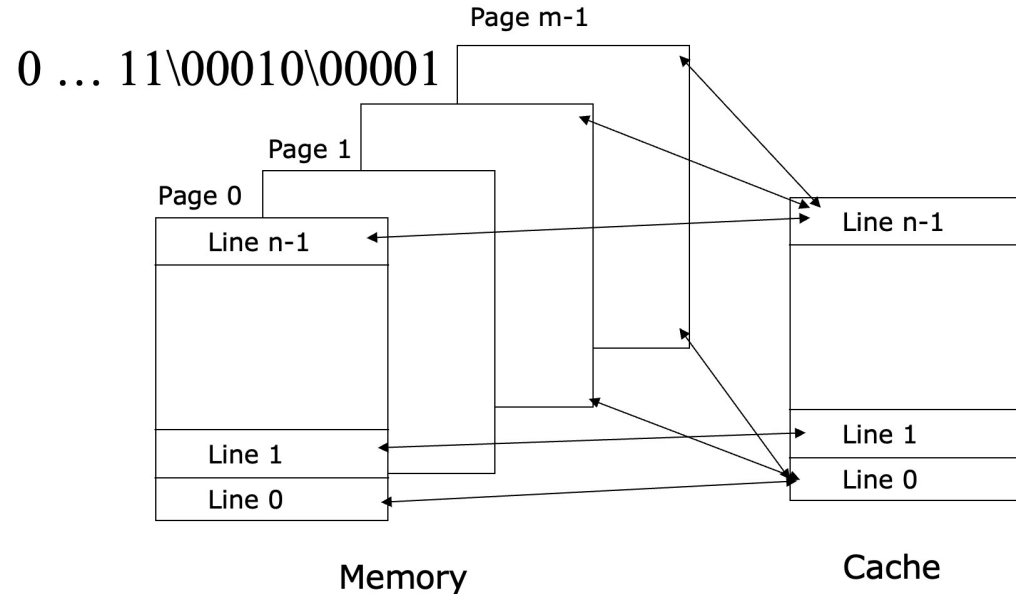
- Temporal locality principle: If the CPU has just accessed data or an instruction, it is highly likely that data or an instruction will be accessed again in the near future.



- ❖ Three ways to organize cache: direct mapping, full associative mapping, and associative set mapping.

❖ Direct mapping:

- Cache: Divided into n blocks or lines from Line 0 to Line $n-1$
- Memory:
 - Divided into m pages, from page 0 to page $m-1$
 - Each memory page has the same size as the cache
 - Each page has n lines, from Line 0 to Line $n-1$
- Mapping:
 - Line i of (page 0 to page $m-1$) is mapped to Line i of the cache



❖ The address field of a direct mapping:

- Tag (bit): is the address of the page in memory
- Line (bit): is the address of the line in a cache page
- Word (bit): is the address of the word in a line

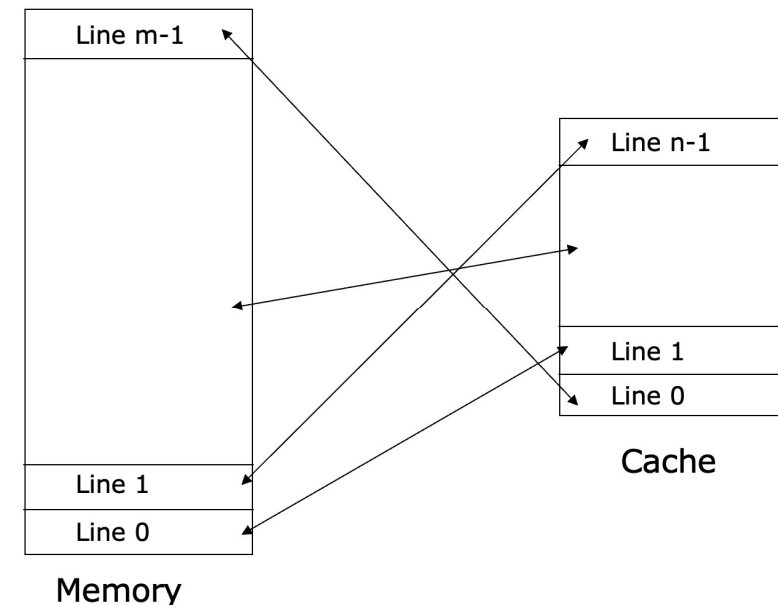
Tag	Line	Word
-----	------	------

How to determine:

- $Tag + Line + Word = Address_Bus_Size(bit) = \log_2 \left(\frac{M_{Memory}}{[Cell]} \right)$
- Number of memory pages: $P = \frac{M_{Memory}}{[M_{Cache}]} \Rightarrow tag = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{Cache}}{[Line]} \Rightarrow line = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{Line}{[Cell]} \Rightarrow word = \log_2 (W)$

❖ Fully associative mapping

- Cache: Divided into n blocks or lines from Line 0 to Line $n-1$
- Memory:
 - Divided into m lines, from Line 0 to Line $m-1$
 - The size of each cache line is equal to the size of a memory line
 - The number of lines in memory can be much larger than the number of cache lines ($m \gg n$)
- Mapping: Line i of memory is mapped to any Line j of cache



❖ The address field of Fully associative mapping

- Tag (bit): is the address of the line in memory
- Word (bit): is the address of the word in memory within a line



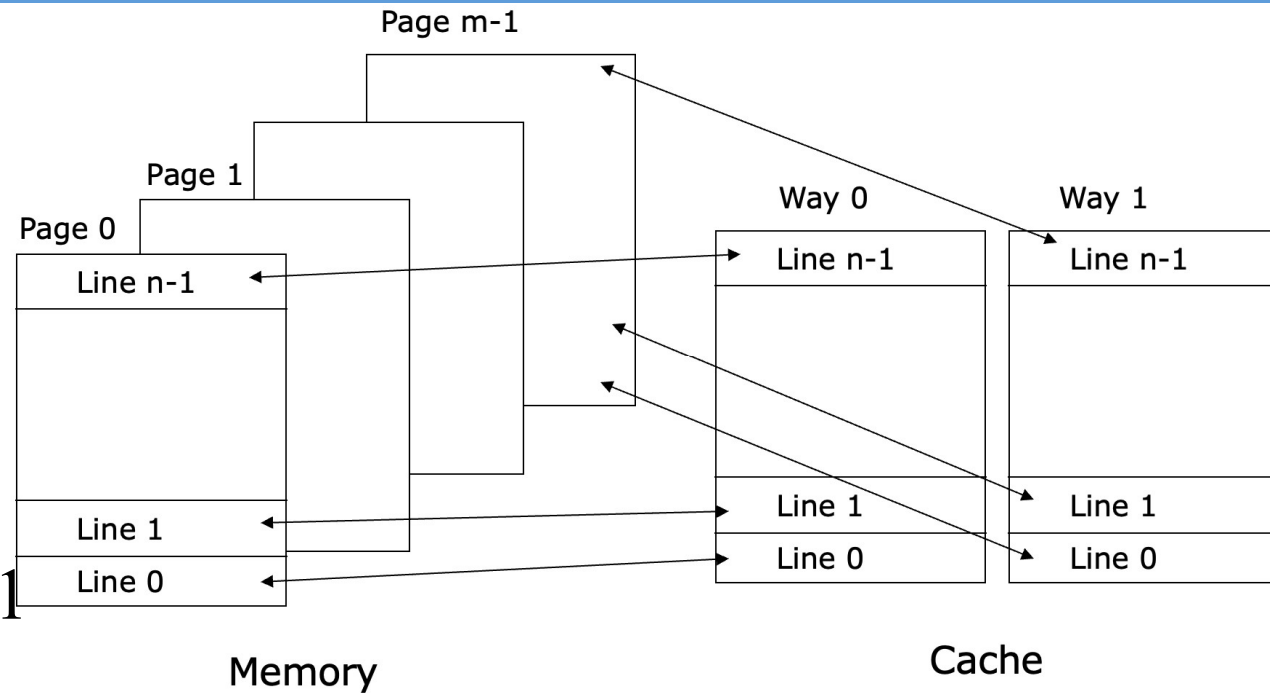
Where

- $\text{Tag} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{Memory}}{[Cell]} \right)$
- Number of memory lines: $L = \frac{M_{Memory}}{[Line]} \Rightarrow tag = \log_2 (L)$
- Number of memory cells in a line:

$$W = \frac{[Line]}{[Cell]} \Rightarrow Word = \log_2 (W)$$

❖ Set associative mapping

- Divided into k ways of equal size
- Each way is divided into n blocks or lines, from Line 0 to Line $n-1$
- Memory:
 - Divided into m pages, from page 0 to page $m-1$
 - The size of one page is equal to the size of one way of the cache
 - Each page is divided into n blocks or lines, from Line 0 to Line $n-1$



❖ Set associative mapping

- Mapping: Pages are mapped to Ways (full mapping):
- A memory page can be mapped to any way in the cache.
 - Page lines are mapped to way lines (direct - fixed mapping):
- Line i of page i is mapped to Line i of way j ;

Tag	Set	Word
-----	-----	------

❖ The address field of Set associative mapping

- Tag (bit): is the address of the page in memory
- Set (bit): is the address of the line in a memory page
- Word (bit): is the address of the word in a memory line

Tag	Set	Word
-----	-----	------

where

- $\text{Tag} + \text{Set} + \text{Word} = \text{Address_Bus_Size (bit)} = \log_2 \left(\frac{M_{\text{Memory}}}{[\text{Cell}]} \right)$
- Number of memory pages: $P = \frac{M_{\text{Memory}} \times \text{Way}}{M_{\text{Cache}}} \Rightarrow \text{tag} = \log_2 (P)$
- Number of lines in a page: $L = \frac{M_{\text{Cache}}}{[\text{Word}] \times \text{Way}} \Rightarrow \text{Set} = \log_2 (L)$
- Number of memory cells in a line: $W = \frac{[\text{Line}]}{[\text{Cell}]} \Rightarrow \text{Word} = \log_2 (W)$

❖ Exercise: Find the size of the address field using Fully associative mapping, knowing:

- Memory size: 4GB
- Cache size: 1KB
- Size of a line: 32 bytes

Solution:

- [Memory] = 4GB = 2^{32} B $\Rightarrow$ Tag + Word = 32 bits
- [Cache] = 1KB = 2^{10} B
- [Word] = 32 bytes = 2^5
- Number of memory lines: $L = [\text{Memory}]/[\text{Word}] = 2^{27} \Rightarrow \text{tag} = 27 \text{ bits}$
- Number of memory cells per line: $W = [\text{Word}]/[\text{Cell}] = 2^5$
 $\Rightarrow \text{Word} = 5 \text{ bits}$

❖ Exercise: Find the size of the address field using direct mapping, given:

- Memory size: 4GB
- Cache size: 1KB
- Line size: 32 bytes

Solution:

- $[\text{Memory}] = 4\text{GB} = 2^2 * 2^{30}\text{B} = 2^{32}\text{B} \Rightarrow \text{Tag} + \text{Line} + \text{Word} = 32 \text{ bits}$
- $[\text{Cache}] = 1\text{KB} = 2^{10}\text{B}$
- $[\text{Word}] = 32 \text{ bytes} = 2^5\text{B}$
- Number of lines: $L = [\text{Cache}]/[\text{Line}] = 2^5 \Rightarrow \text{line} = 5 \text{ bits}$
- Number of memory cells per line: $W = [\text{Line}]/[\text{Cell}] = 2^5\text{B}/1\text{B} = 2^5$
 $\Rightarrow \text{Word} = 5 \text{ bits}$
- Number of memory pages: $P = [\text{Memory}]/[\text{Cache}] = 2^{22} \Rightarrow \text{tag} = 22 \text{ bits}$
or $\text{tag} = 32 - \text{Line} - \text{Word} = 22.$

❖ Exercise: Find the size of the address field using associative set mapping:

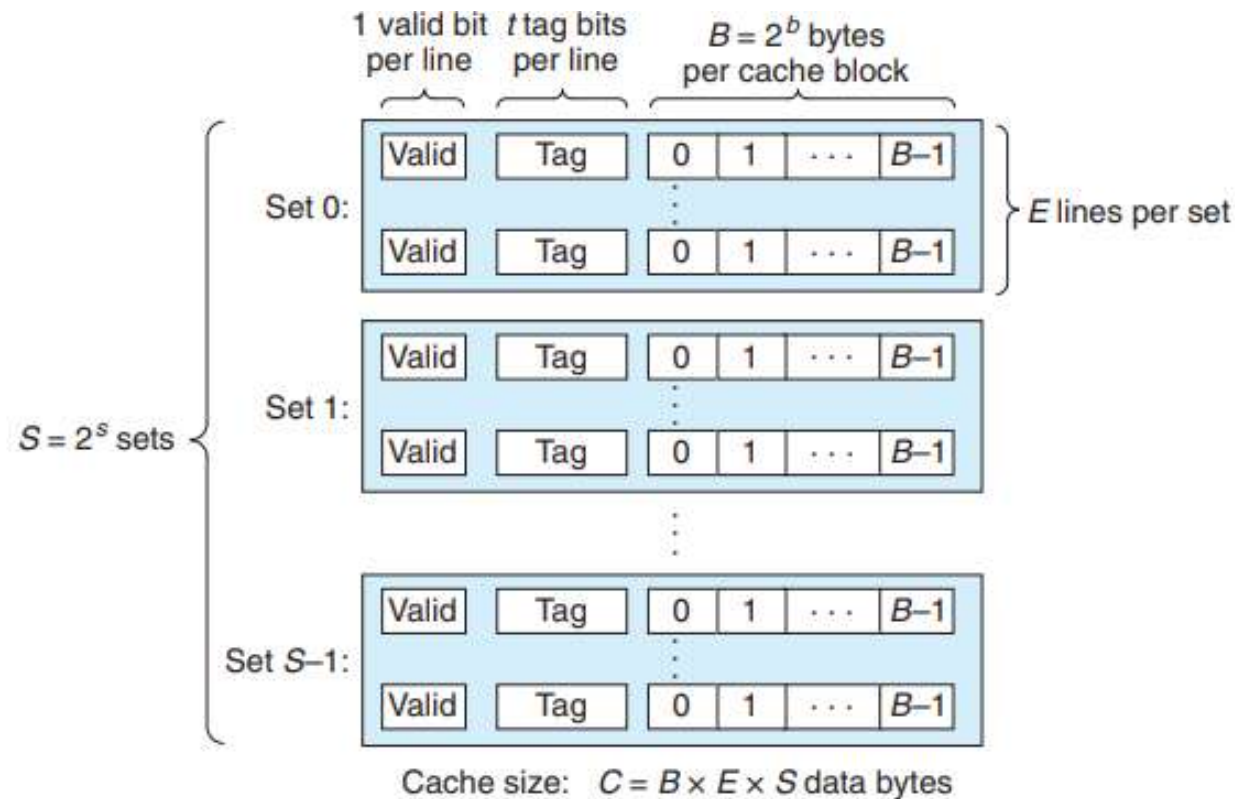
- Memory size: 4GB
- Cache size: 1KB, 2 ways
- Size of a line: 32 bytes

Solution:

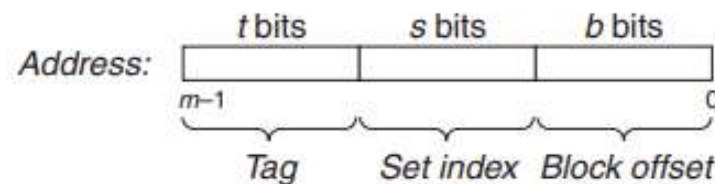
- $[Memory] = 4GB = 2^2 * 2^{30}B = 2^{32}B \Rightarrow Tag + Line + Word = 32 \text{ bits}$
- $[Cache] = 1KB = 2^{10}B, \text{ way} = 2$
- $[Word] = 32 \text{ bytes} = 2^5B$
- Number of memory pages: $P = [Memory] \times [Way] / [Cache] = 2^{23}$
 $\Rightarrow \text{tag} = 2^3 \text{ bits}$
- Number of lines: $L = [Cache] / ([Word] \times [Way]) = 2^4 \Rightarrow \text{set} = 4 \text{ bits}$
- Number of memory cells per line: $W = [Word] / [Cell] = 2^5$
 $\Rightarrow \text{Word} = 5\text{bit}$

- ❖ Calculating the number of bits in the cache:
 1. Calculate the total number of bits required for a direct-mapped cache memory of size 64 KByte and a cache line of size 1 memory word, assuming a 32-bit address bus?
 2. What is the total number of bits needed for a cache memory using a 4-way associative set of size 64 KByte and a cache line of size 1 memory word, assuming a 32-bit address bus?
 3. Calculate the total number of bits required for a direct-mapped cache memory of size 64 KB and a cache line of size 8 memory words, assuming a 32-bit address bus?

❖ Calculating the number of bits in the cache:



(a)



- ❖ Calculate the total number of bits required for a direct-mapped cache with a size of 64 KByte of data and a cache line size of 1 memory word, assuming a 32-bit address bus?

Solution:

- We have 1 cache line of 32 bits = 4 bytes $\Rightarrow$ word = 2 bits
- The number of cache lines is $L = [\text{Cache}]/[\text{Cell}] = 64 \text{ Kbytes}/4 \text{ bytes} = 2^{14}$
 $\Rightarrow \text{line} = 14 \text{ bits}$
- $\text{tag} = \text{address_size} - \text{line} - \text{word} = 32 - 14 - 2 = 16 \text{ bits}$
- The number of bits required for one cache line is:
 $\text{bits/block} = \text{data bits} + \text{tag bits} + \text{valid bit} = 32 + 16 + 1 = 49 \text{ bits}$
- The total number of bits required for the cache memory is $= L \times \text{bits/block}$
 $= 2^{14} \times 49 \text{ bits} = 98 \text{ Kbytes}$

❖ What is the total number of bits required for a cache using a 4-way associative array with a data size of 64 KByte and a cache line size of 1 memory word, assuming a 32-bit address bus?

Solution:

- We have 1 cache line of 32 bits = 4 bytes $\Rightarrow$ word = 2 bytes
- The number of cache lines is $L = [\text{Cache}]/[\text{Word}] * \text{Way} = 64 \text{ Kbytes} / 4 \text{ bytes} * 4 = 2^{12} \Rightarrow \text{line} = 12 \text{ bits}$
- $\text{tag} = \text{address_size} - \text{line} - \text{word} = 32 - 12 - 2 = 18 \text{ bits}$
- The number of bits required for one cache line is:
- $\text{bits/block} = \text{data bits} + \text{tag bits} + \text{valid bit} = 32 + 18 + 1 = 51 \text{ bits}$
- The total number of bits required for the cache memory is $= 4 \times L \times \text{bits/block} = 2^{14} \times 51 \text{ bits} = 102 \text{ Kbytes}$

❧ Calculate the total number of bits required for a direct-mapped cache of size 64 KB and a cache line of size 8 memory words, assuming a 32-bit address bus?

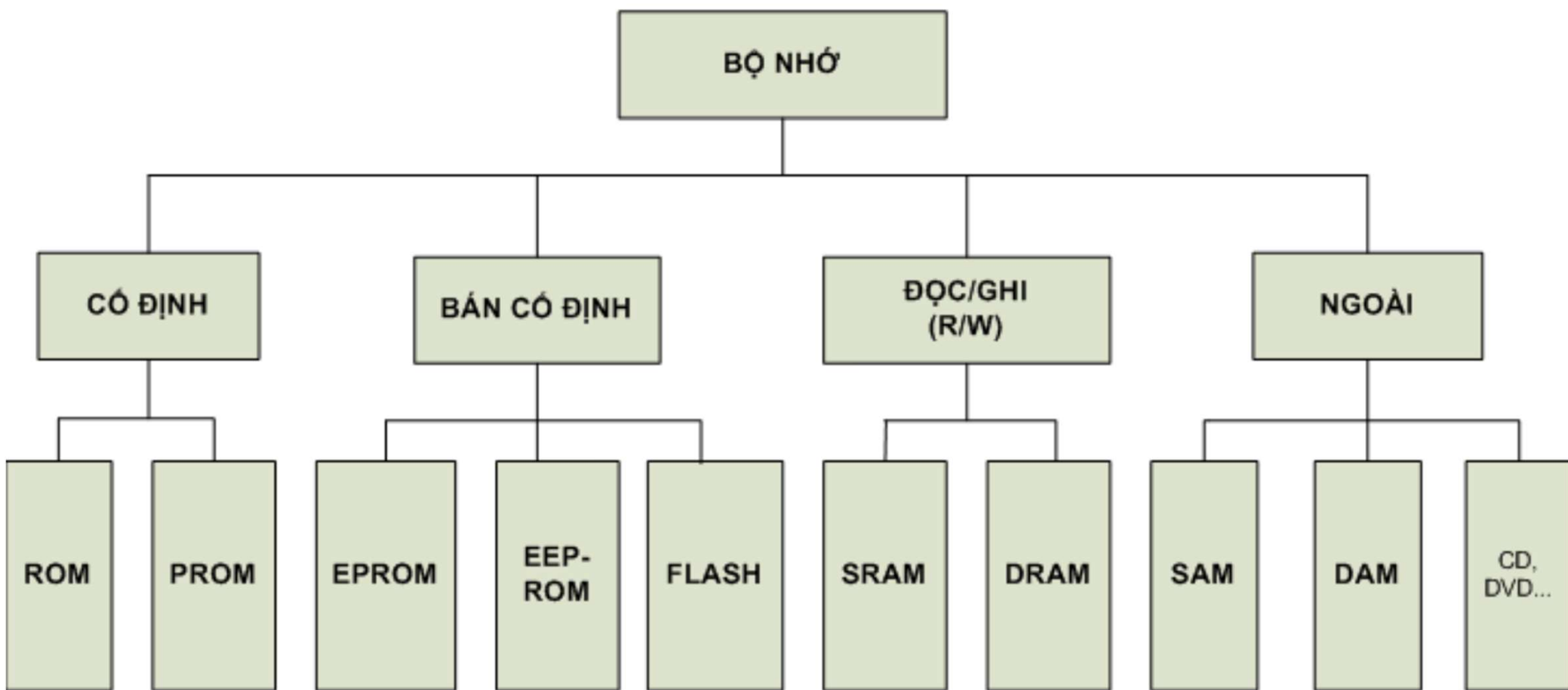
Solution:

- We have 1 cache line $8 \times 32 \text{ bits} = 32 \text{ bytes} \Rightarrow \text{word} = 5 \text{ bits}$
- The number of cache lines is $L = [\text{Cache}]/[\text{Word}] = 64 \text{ Kbytes}/32 \text{ bytes} = 2^{11} \Rightarrow \text{line} = 11 \text{ bits}$
- $\text{tag} = \text{address_size} - \text{line} - \text{word} = 32 - 11 - 5 = 16 \text{ bits}$
- The number of bits required for one cache line is:
$$\text{bits/block} = \text{data bits} + \text{tag bits} + \text{valid bit} = 8 \times 32 + 16 + 1 = 273 \text{ bits}$$
- The total number of bits required for the cache memory is $= L \times \text{bits/block} = 2^{12} \times 273 \text{ bits} = 68.25 \text{ Kbytes}$

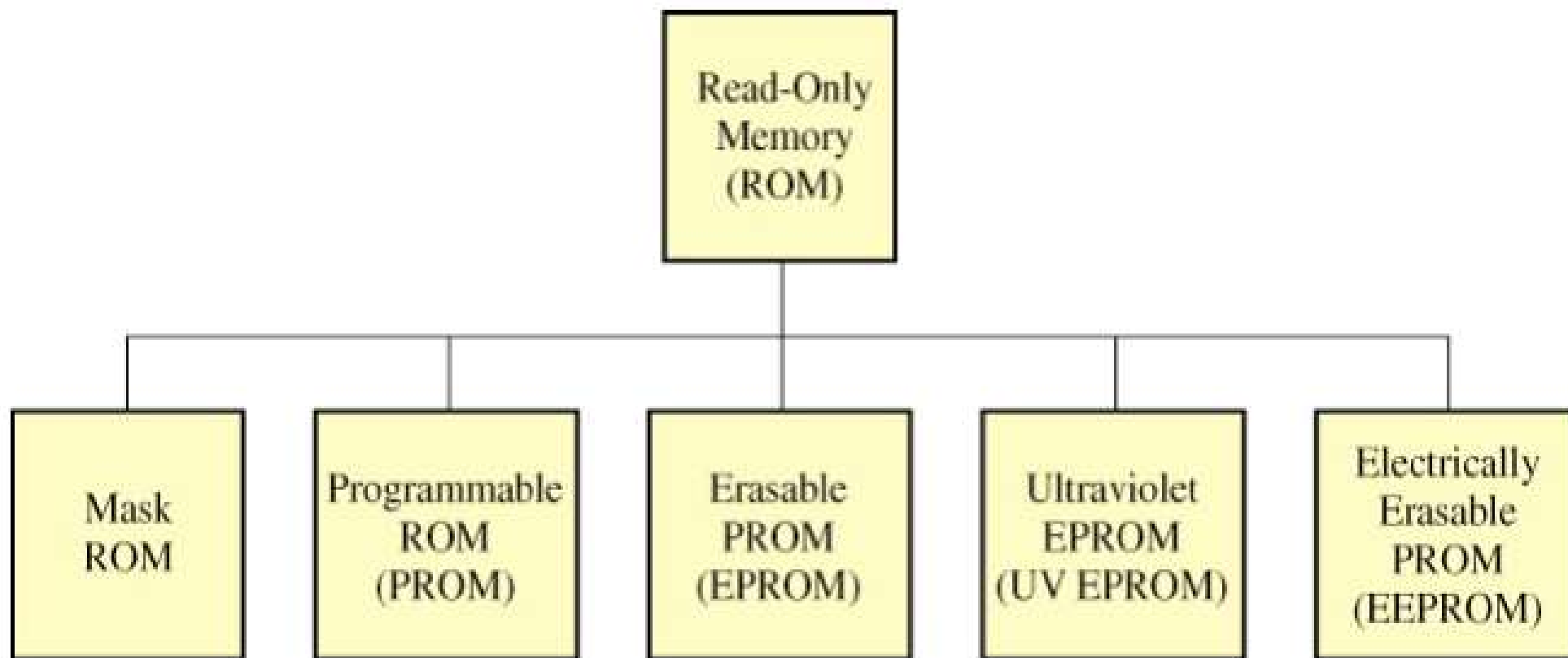
- ❖ Exercise: Given a computer with 64 Kbytes of byte-addressed main memory and a cache consisting of 32 lines organized as direct mappings, each line measuring 8 bytes.
1. Determine the number of bits in the address fields: Tag, Line, Word.
 2. Indicate which cache line each main memory byte with the address below is loaded into:

0001 0001 0001 1011
1100 0011 0011 0100
1101 0000 1101 1101
1010 1010 1010 1010
 3. Assuming the memory byte with address 0001 1010 0001 1010 is loaded into the cache, indicate the binary addresses of the other memory bytes loaded with that byte on the same line.

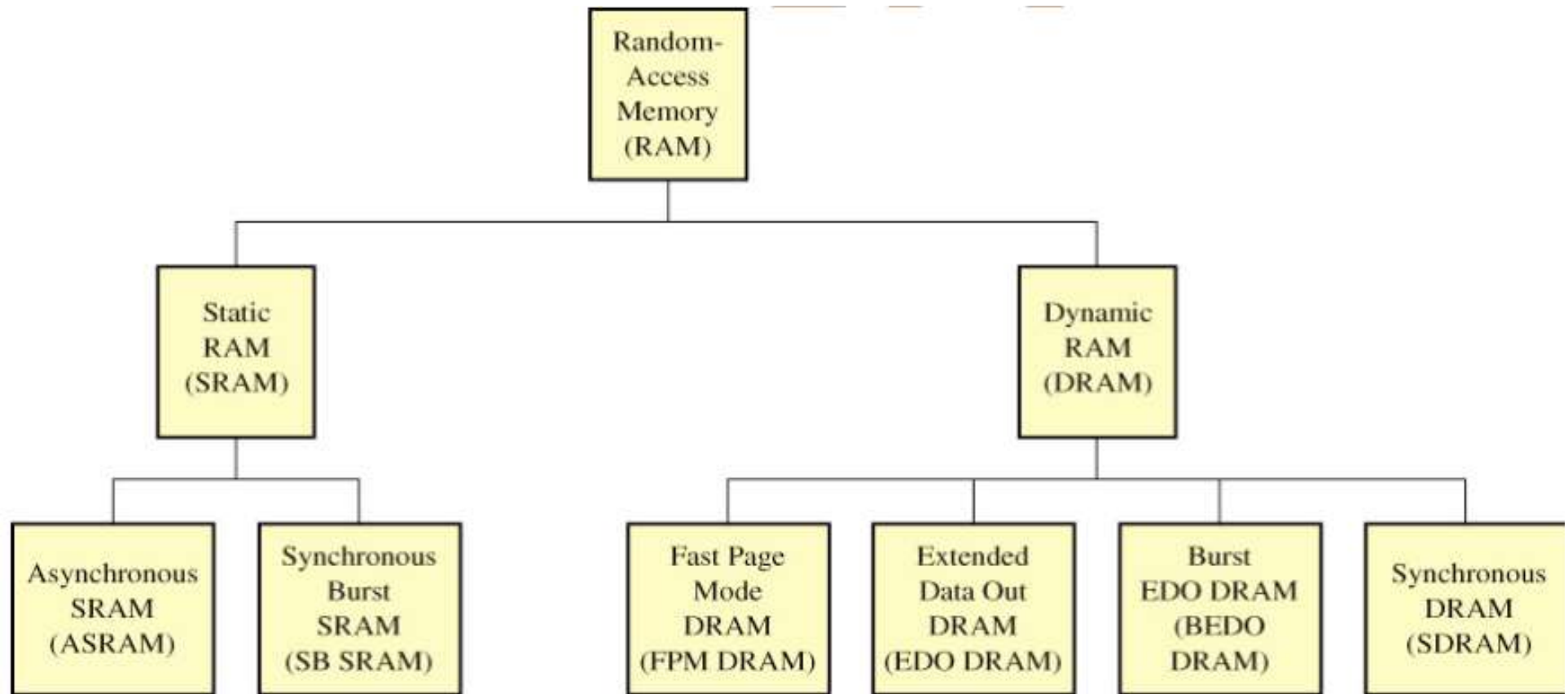
❖ Classification:



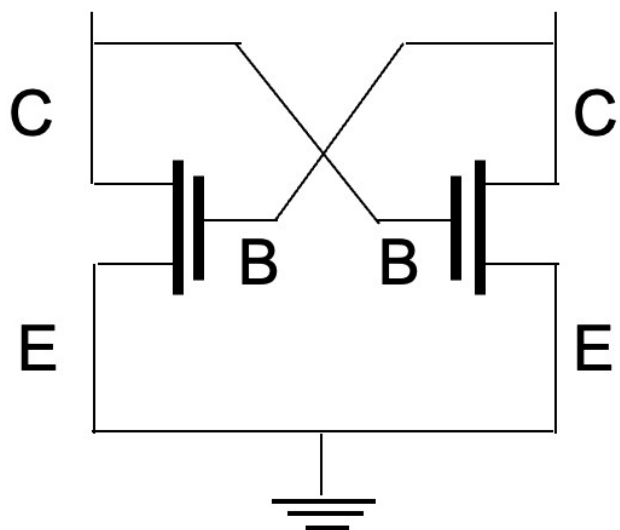
❖ ROM:



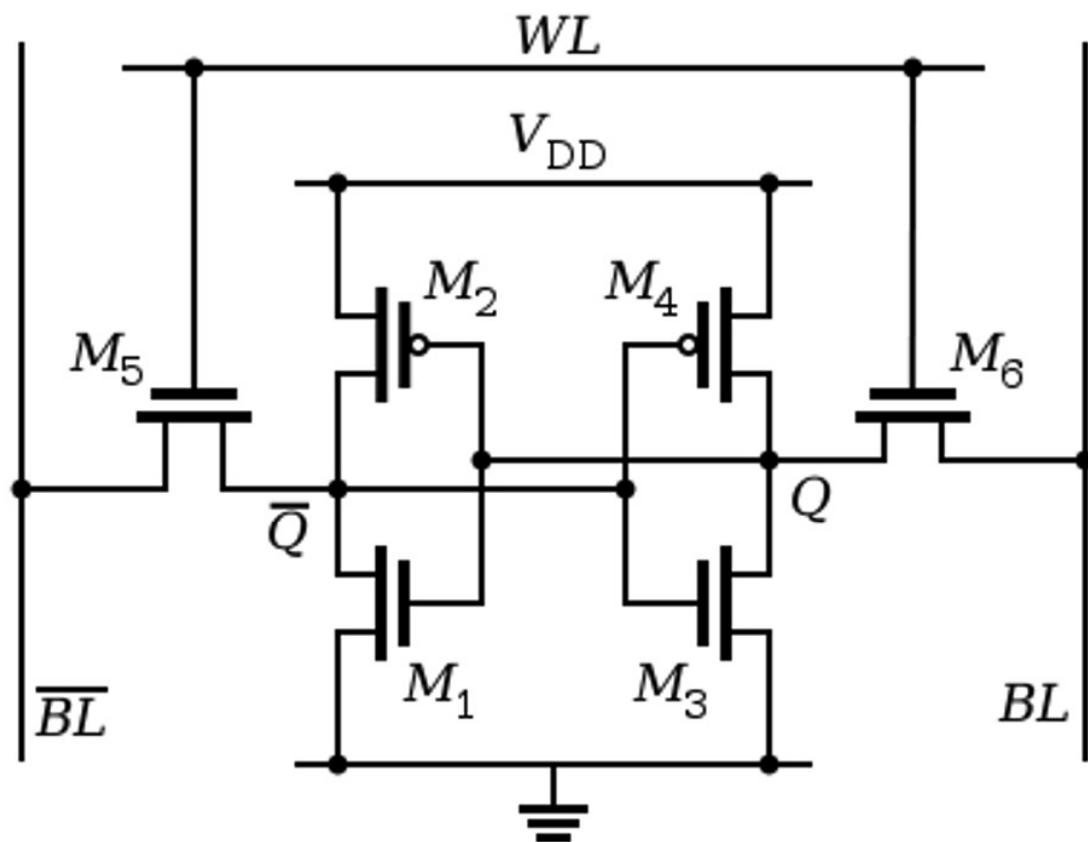
❖ RAM:



❖ SRAM: Expensive, often used as a cache.

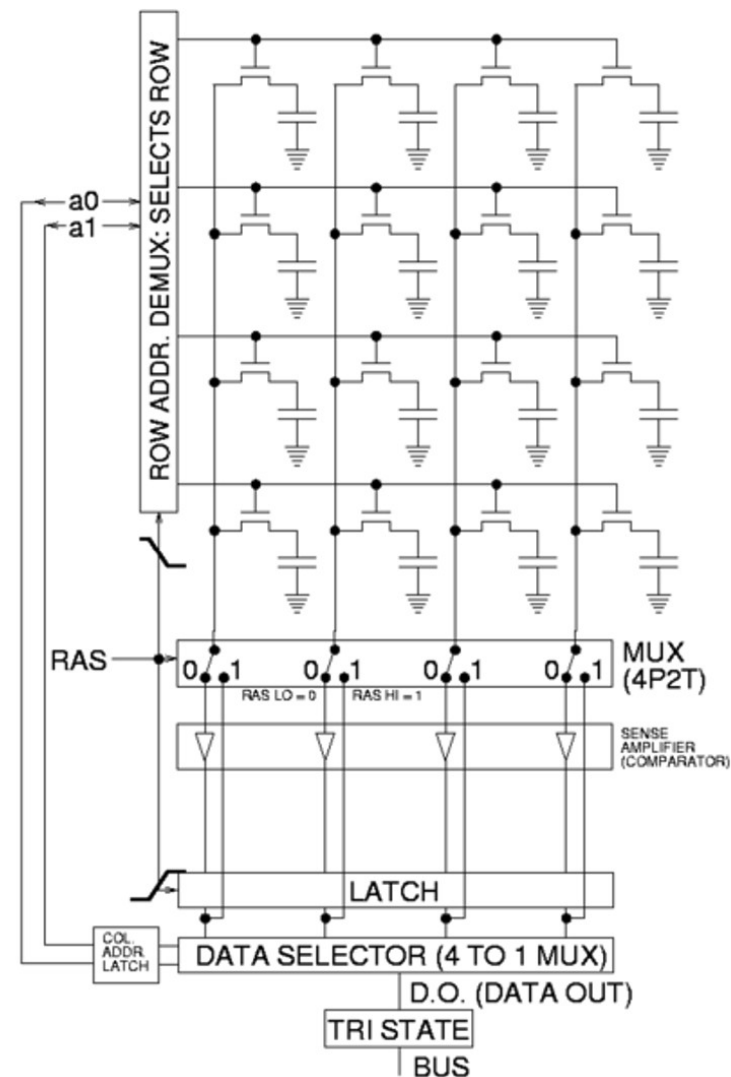
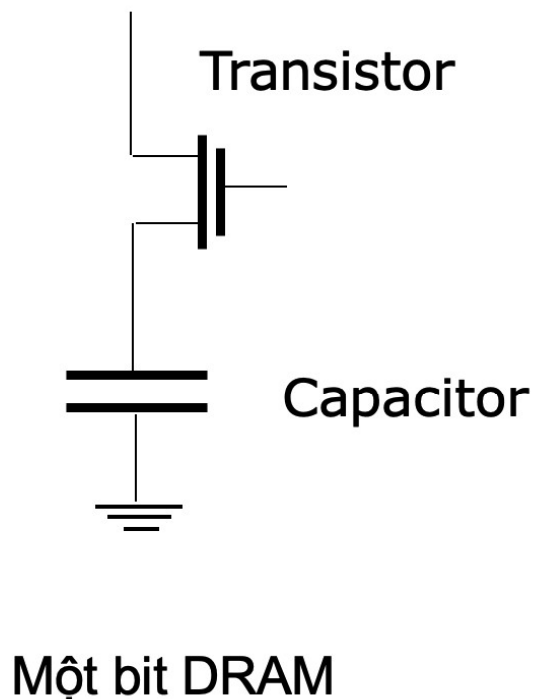


Một mạch lật đơn giản



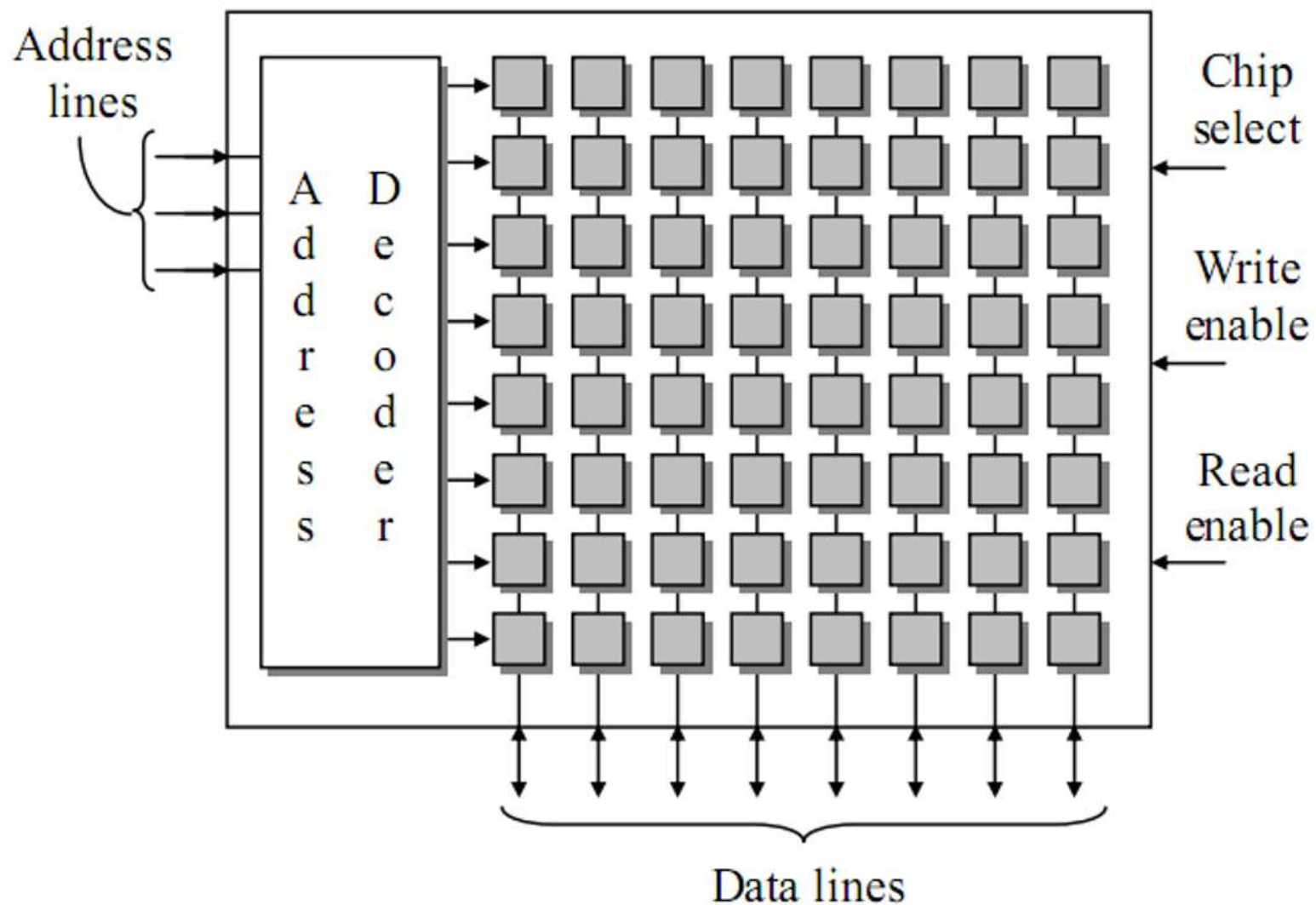
Một ô nhớ SRAM loại 6T

❖ DRAM: Cheaper cost, requires a refresh circuit.



- ❖ System Bus
 - Address Bus → select memory location
 - Data Bus → transfer data
 - Control Bus → read/write, interrupt
- ❖ When MCU lacks RAM/Flash: use external SRAM/external Flash
 - Required :
 - address decoding,
 - chip select signal,
 - appropriate timing

Memory Organization



❖ Address lines:

- Address lines connected to bus A
- Transmit address signals from the CPU to the memory circuit

❖ Address decoder:

- Address Decoder
- Uses an address to select and activate the memory location/row to be accessed

❖ Data lines:

- Data lines connect to the D bus
- Transmits data from memory to the CPU and vice versa

❖ Chip select CS:

- Chip select signal pin
- The memory chip is activated when $CS=0$. Normally, the CPU only works with one memory chip at a time

❖ Write enable WE:

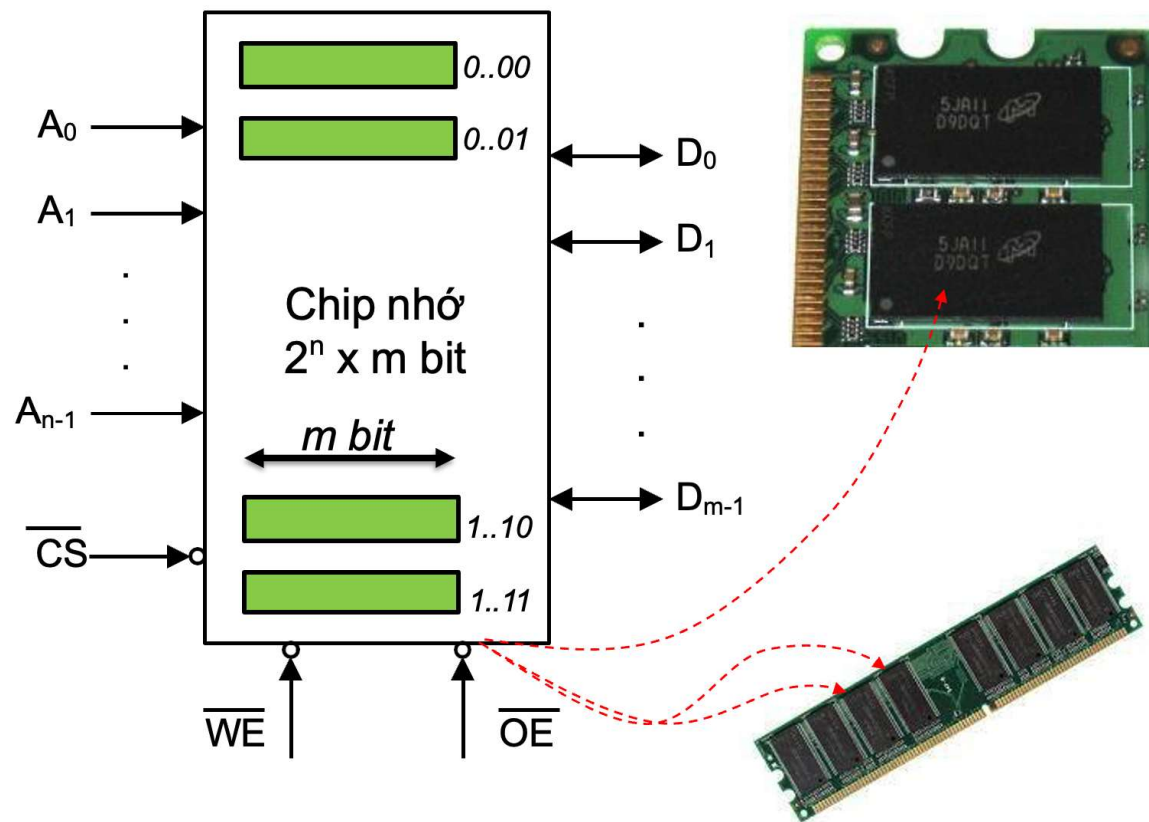
- Write enable signal pin
- Enables writing to memory line when $WE = 0$

❖ Read enable RE:

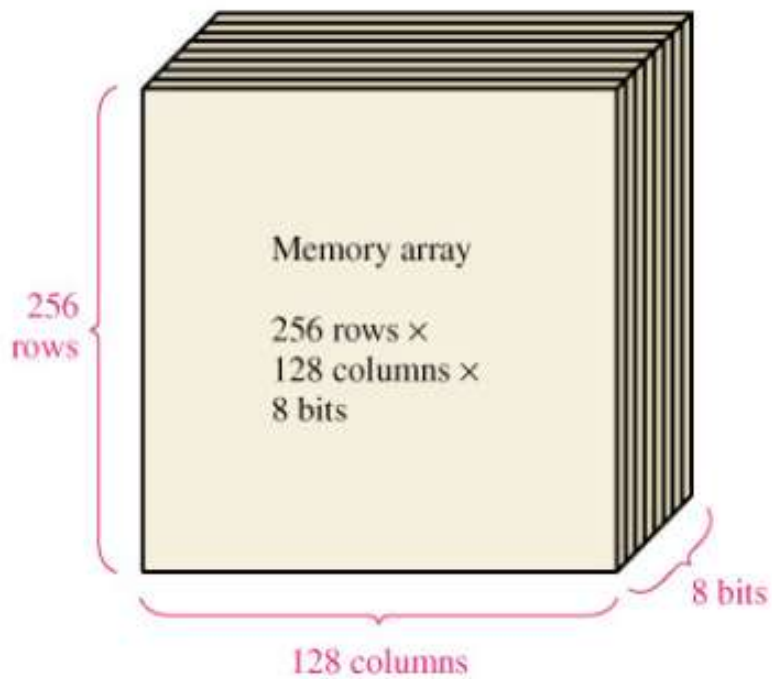
- Read Enable Signal Pin
- Allows reading data from the memory line when $RE = 0$

❖ Basic diagram of a memory chip:

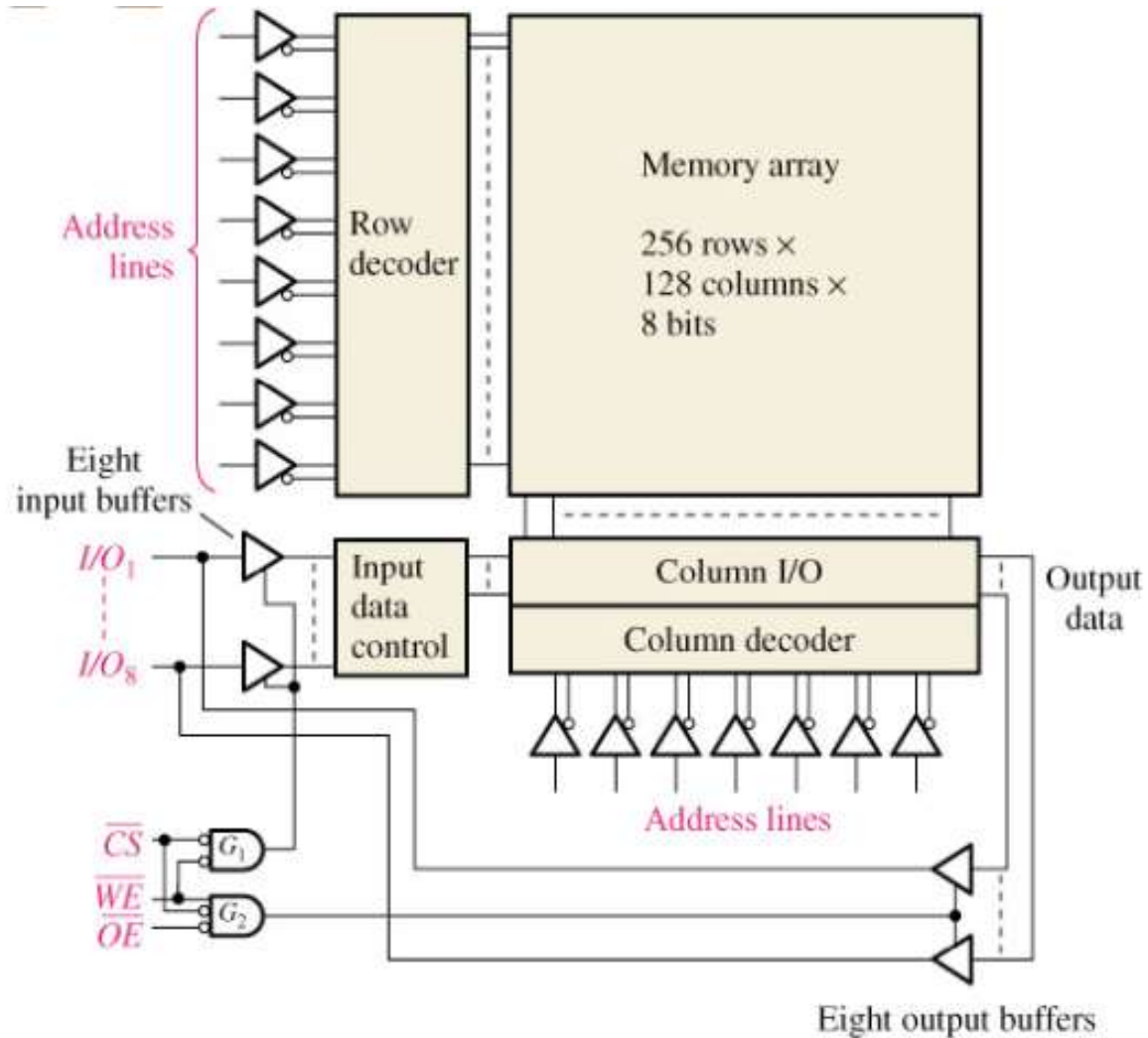
- Address lines: $A_{n-1} - A_0$
 $\rightarrow 2^n$ memory words
- Data lines: $D_{m-1} - D_0$
 $\rightarrow$ memory word length = m bits
- Memory chip capacity
 $\rightarrow 2^n \times m$ bits



Memory Organization



(a) Memory array configuration



(b) Memory block diagram

❖ Memory design

The physical memory design process involves combining discrete memory chips to create the memory capacity and word length required by the CPU.

- A memory chip is a specific microchip, arranged with basic pins.
- The pins of a typical memory chip include address bus inputs, data inputs, chip select control pins, write/read and power pins.

❖ Memory Design Steps

1. Schematic design: Use address decoder circuits (such as SN74138) to determine the memory areas of the board within the CPU's address space.
2. Control signal connection: Connect the RD (Read), WR (Write) lines and the corresponding data BUS system between the CPU and the memory chips.
3. Address partitioning: Establish an address table (Memory Map) to manage which areas are allocated for ROM (containing BIOS/OS) and which areas are allocated for RAM (data/applications).

❖ Memory design

- Designing memory from available memory chips has the following steps:

1. Determine the number of memory chips to create the required memory capacity according to the formula:

$$M = Q/D, \text{ whereas}$$

Q is the capacity of memory.

D is the capacity of each chip

M is the number of memory chips needed.

2. Determine the number of base address wires (ie the number of low address wires connected directly to the memory chip or interconnect chip): according to the following expression:

$$2^m = D, \text{ whereas}$$

m is the wire number of the base address.

D is the capacity of each chip

3. Determine the number of address lines to create chip select according to the following expression:

$$2^i = M, \text{ whereas}$$

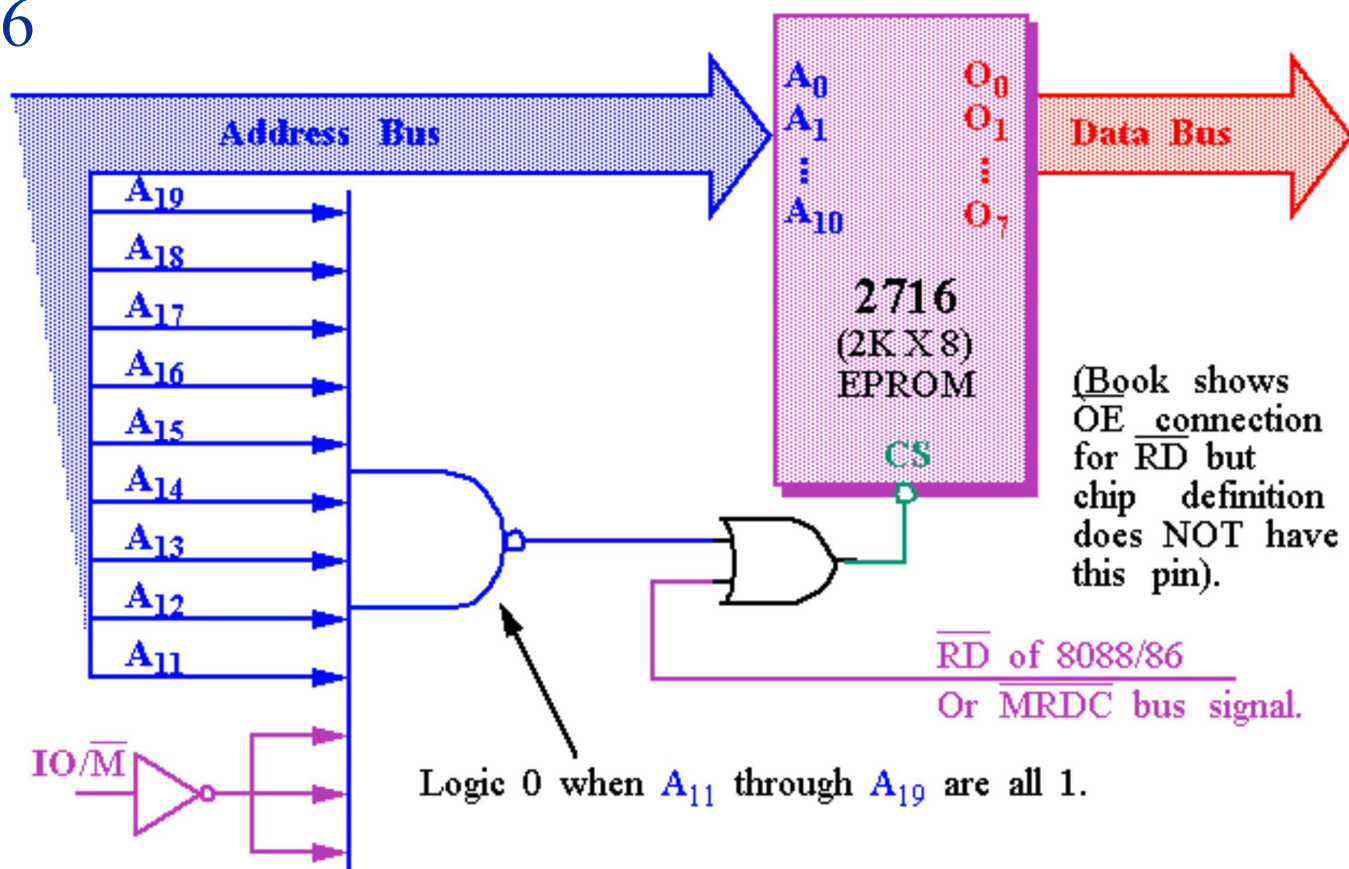
D is the capacity of each chip

i is the number of wires needed to decode the signals chip select signal (Csi)

- ❖ General Address Decoding Steps: Building an address decoding circuit for a ROM memory with capacity C_{IC} ; Knowing that the size of a memory IC is IC and the base address is DCCS
 - Step 1: Determine the number of bits for the internal chip address and the decoding circuit.
 - The number of bits for the internal chip is $m = \log_2 IC \Rightarrow$ the number of bits needed for the decoding circuit is $20 - m$
 - Step 2: Resolve the base address of the chips.
 - The number of memory chips is $n = C_{IC}/IC \Rightarrow$ The capacity of one memory chip is IC.
 - A memory chip has: End address = Start address + Chip capacity - 1.
 - Address of IC 1: From DCCS to $(DCCS + IC - 1H)$.
 - Address of IC 2: From $(DCCS + IC)$ to $(DCCS + 2 \times IC - 1H)$
 - ...
 - Step 3: Create a bit table
 - Step 4: Draw the decoding circuit diagram

Memory design

- ❖ Decoding memory addresses using basic logic circuits: Decoding memory addresses using basic logic circuits: Build an address decoder circuit for a 4KB ROM memory using basic logic circuits; knowing that the size of a memory IC is 2Kx8 and the base address is 03800H;
- ❖ Assume using 8086



❖ Decoding memory addresses using basic logic circuits:

CROM = 4KB; IC = 2Kx8; DCCS = 03800H

- Step 1: Determine the number of bits for the internal chip address and decoding circuit
 - A 2Kx8 IC memory chip occupies 2KB of space = $2^1 \times 2^{10} = 2^{11}$ B $\Rightarrow$ 11 bits for the internal chip address (A0 - A10)
 - The 8086 microprocessor has 20 bits of address space, so the number of bits for the decoding circuit is: $= 20 - 11 = 9$ bits for the decoding circuit (A11 - A19)

❖ Decoding memory addresses using basic logic circuits:

$CROM = 4KB$; $IC = 2K \times 8$; $DCCS = 03800H$

- ► Step 2: Resolve the base address of the chips
 - The ROM memory has a CROM capacity of 4KB and each memory chip has a capacity of 2KB $\Rightarrow$ The number of $2K \times 8$ memory chips is $4KB / 2KB = 2$ corresponding to IC1 and IC2
 - Convert the size of a memory chip to base 16:
 - $2KB = 2^{11} = 0000\ 0000\ 1000\ 0000\ 0000B = 00800H$
 - $\Rightarrow$ The capacity of a memory chip is 00800H

❖ Decoding memory addresses using basic logic circuits:

$CROM = 4KB$; $IC = 2K \times 8$; $DCCS = 03800H$

- ► Step 2: Resolve the base address of the chips
 - Address of a memory chip: End address = Start address + Memory chip capacity - 1
 - Address range of IC1:
From 03800H to (03800H + 00800H - 1)
i.e., From 03800H to 03FFFH
 - Address range of IC2:
From 04000H to 047FFFH
 - ...

❖ Decoding memory addresses using basic logic circuits:

$CROM = 4KB$; $IC = 2K \times 8$; ĐCCS = 03800H

- Step 3: Create a bit table from the base addresses of the chips

		9 bit cho mạch giải mã ($A_{19} - A_{11}$)									11 bit địa chỉ nội bộ chip ($A_{10} - A_0$)										
IC1	03800H	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
IC2	04000H	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- Step 4: Draw the address decoding circuit:

		9 bit cho mạch giải mã ($A_{19} - A_{11}$)									11bit địa chỉ nội bộ chip ($A_{10} - A_0$)										
IC1	03800H	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
IC2	04000H	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		A_{19}	A_{18}	A_{17}	A_{16}	A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0

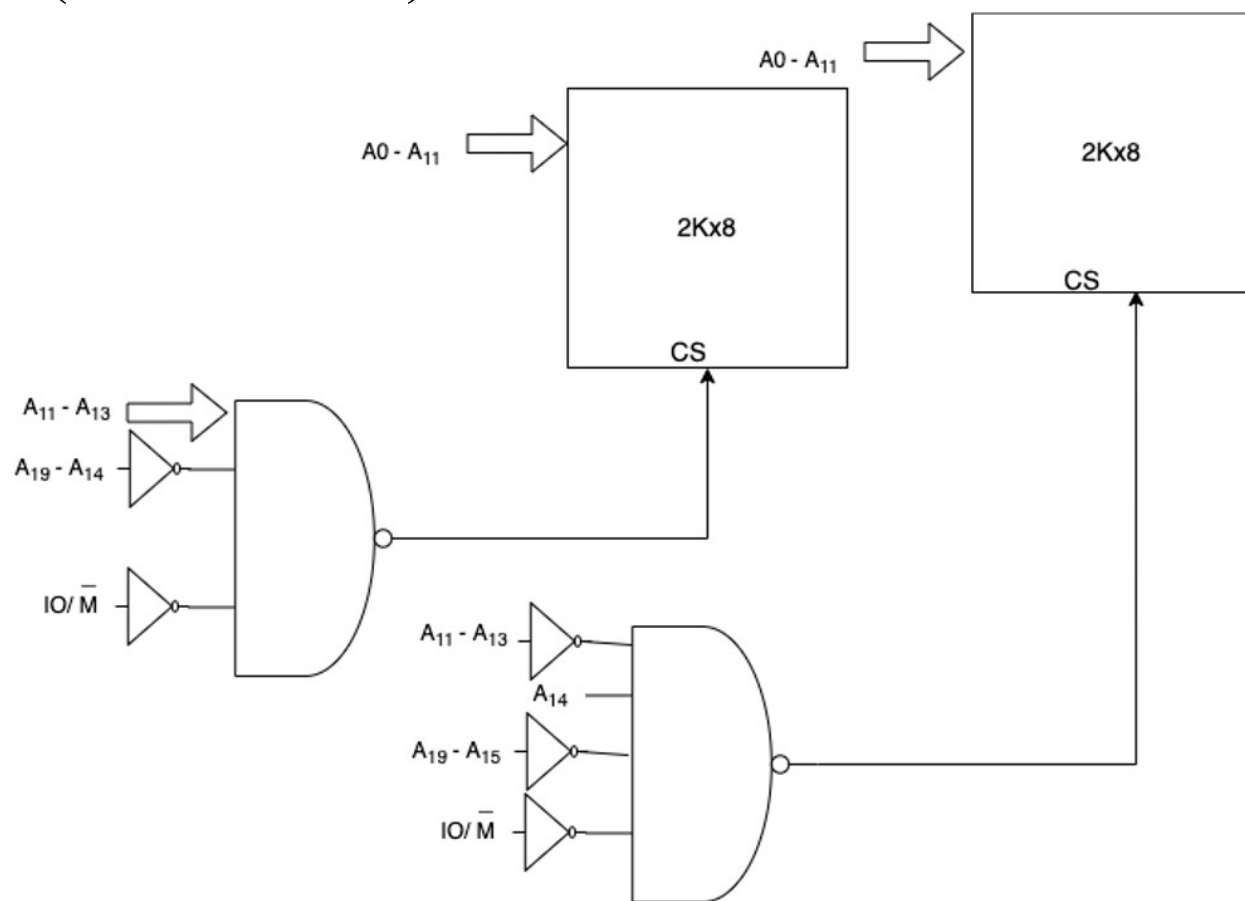
Memory design

❖ Decoding memory addresses using basic logic circuits:

$CROM = 4KB$; $IC = 2K \times 8$; $\text{DCCS} = 03800H$

■ Consider the 9 high bits ($A_{11} - A_{19}$)

- One for 03800H
- One for 04000H



❖ ROM/EPROM Design: Build a 32KB ROM module using an 8K x 8-bit 2764 chip, with the starting address being 20000hex.

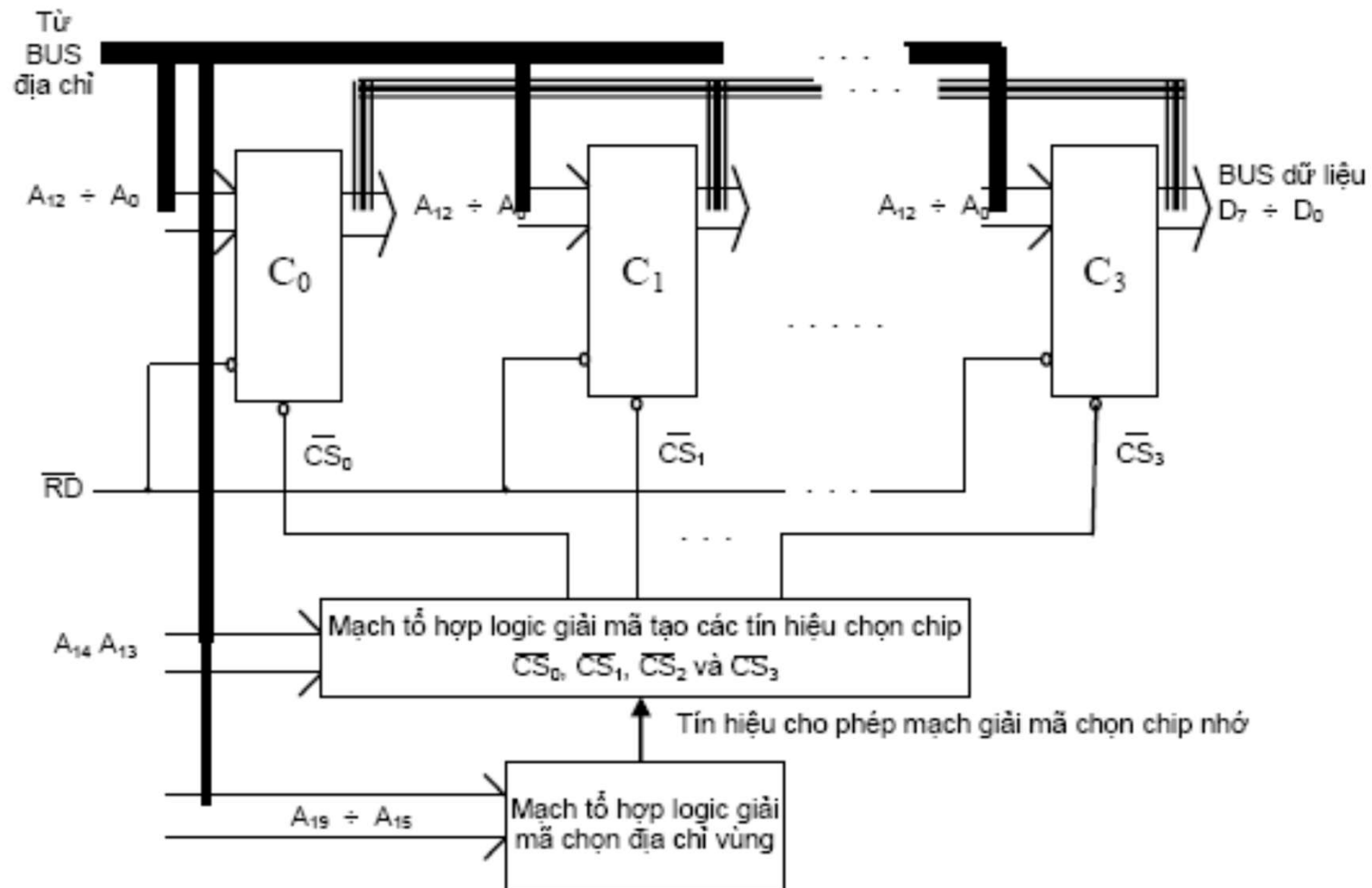
- The x86 CPU uses 20 address lines for the first 1MB: A19 – A0.
- To access the 8K addresses/chip, 13 address lines are needed, from A12-A0.
- To access each of the 4 chips, 2 address lines are needed: A14-A13.

Chip #	Địa chỉ đầu	Địa chỉ cuối	Dung lượng
C0	20000	21FFF	8 K
C1	22000	23FFF	8 K
C2	24000	25FFF	8 K
C3	26000	27FFF	8 K

- To define the address range, 5 address lines are needed: A14-A19 of the first 1MB

Memory design

- ❖ ROM/EPROM Design: Build a 32KB ROM module using an 8K x 8-bit 2764 chip, with the starting address being 20000hex.

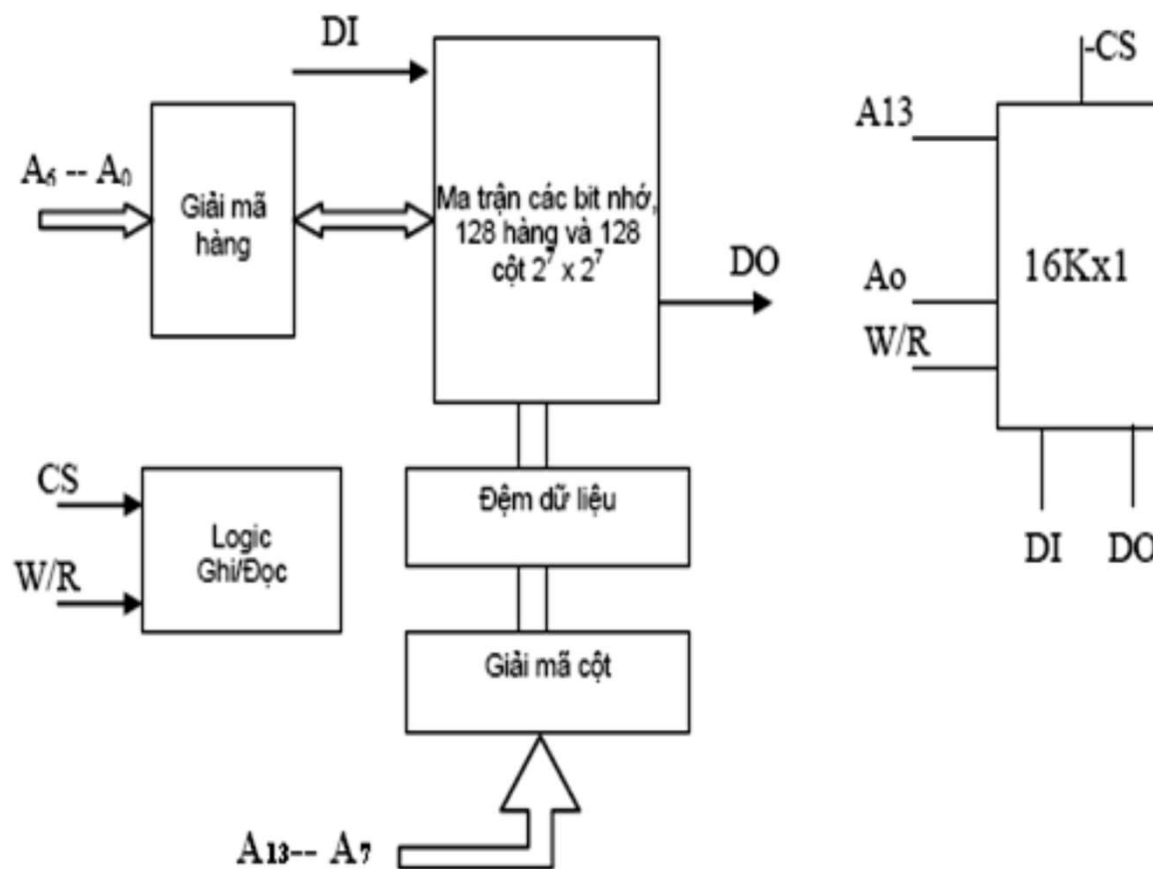


❖ Static RAM Design: A 16Kbyte memory is built using 16Kx1bit SRAM chips. The 16Kbyte SRAM memory band is built on 8 x 16Kx1bit SRAM chips to obtain an 8-bit memory cell.

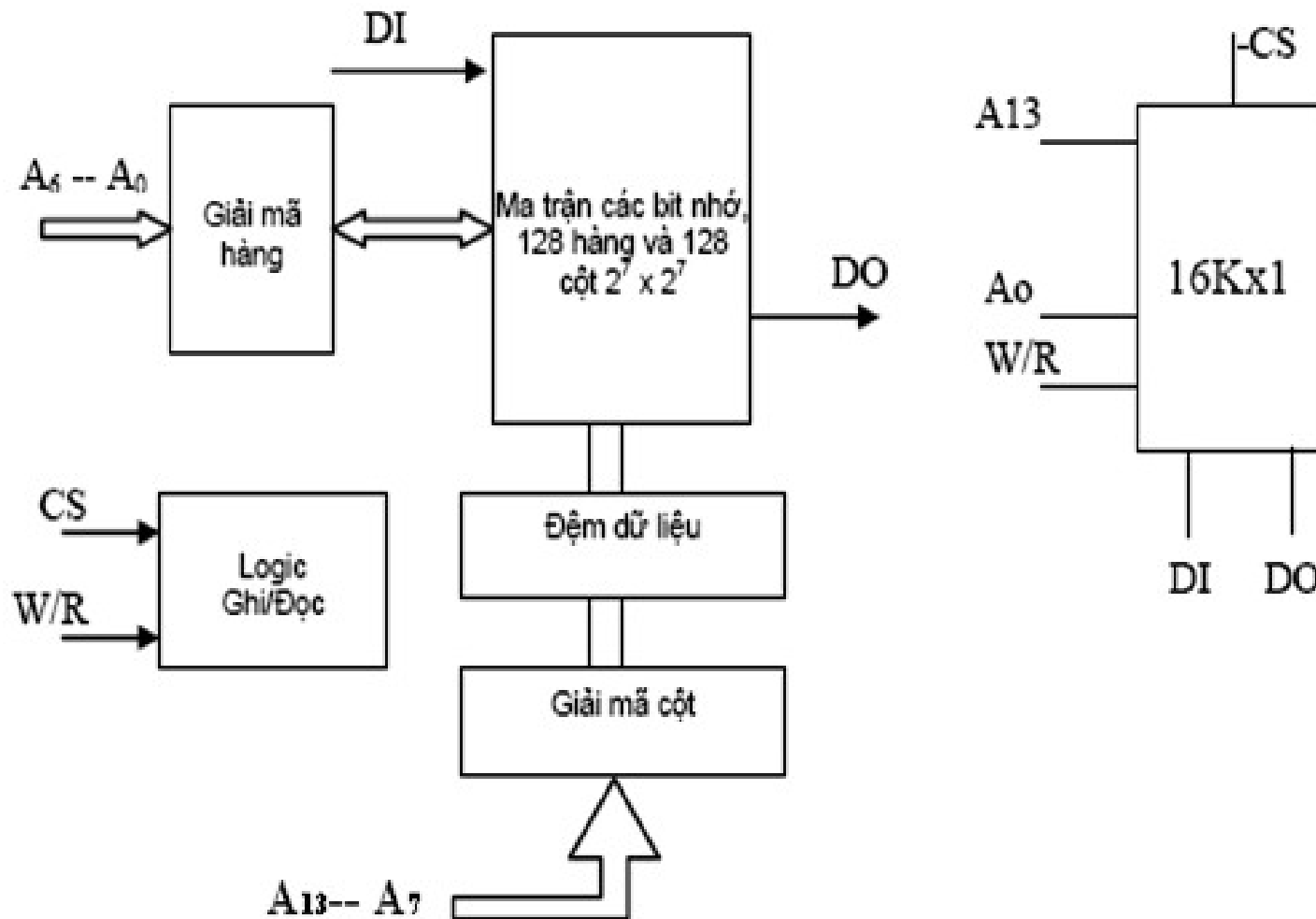
- To achieve this, 8 x 16Kx1bit SRAM chips are arranged so that each chip at a specific location stores a data bit with a corresponding weight within the data byte.
- Signal Lines:
 - A13 - A0 Address Bus, sufficient for 16K (16383) addresses
 - CS: Chip select signal. If CS = 0, the chip is accessible.
 - W/R: Read/write control signal. W=0 (W/) controls recording, and also controls the 3-state integrated circuits in the In/Out directions.
 - Select the memory band to add A14, A14=0 selects the first band, (A14=1 selects the second band).

Memory design

- ❖ Static RAM Design: A 16Kbyte memory is built using 16Kx1bit SRAM chips. The 16Kbyte SRAM memory band is built on 8 x 16Kx1bit SRAM chips to obtain an 8-bit memory cell.



❖ Static RAM Design : loại 16K x 1bit

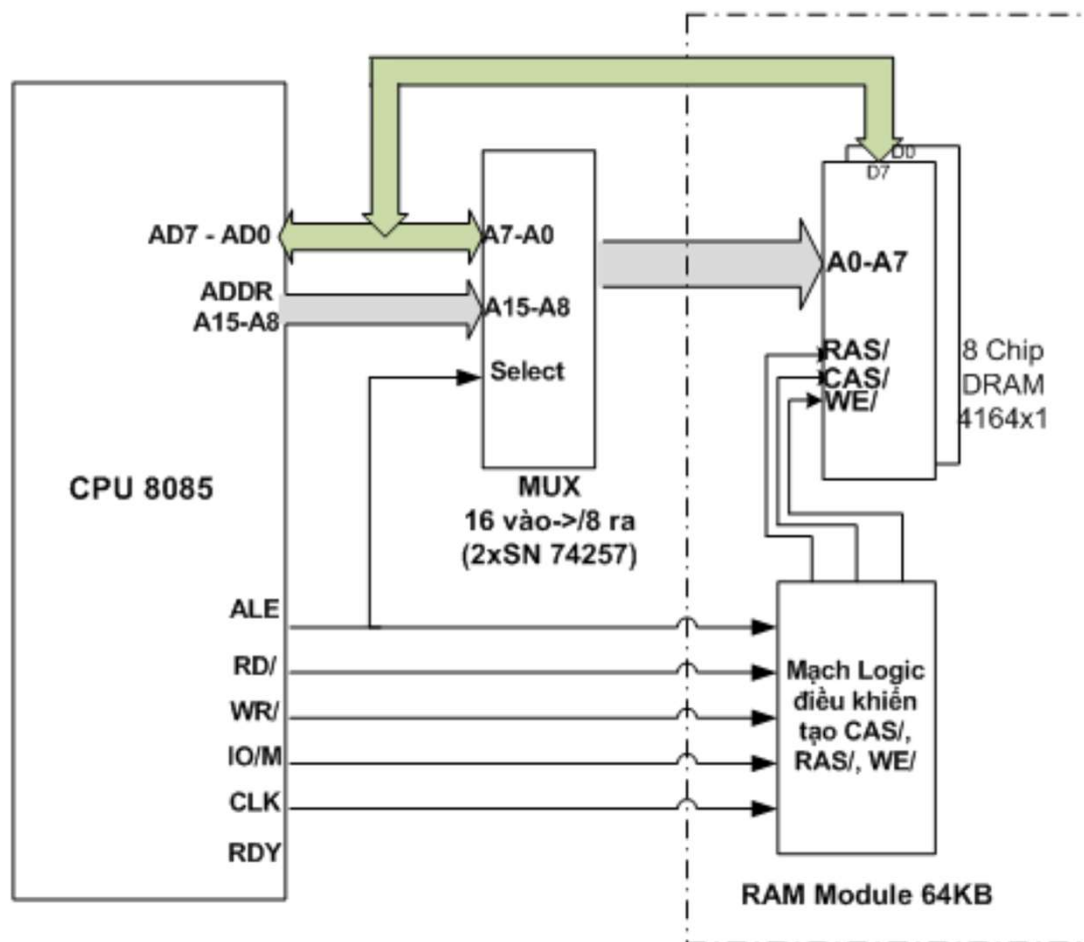


❖ **Dynamic RAM (DRAM) Design:** MK 4164 DRAM is a 64K-bit DRAM on a single chip. The assumption is to design RAM for an 8085 CPU with a maximum RAM of 64 KB.

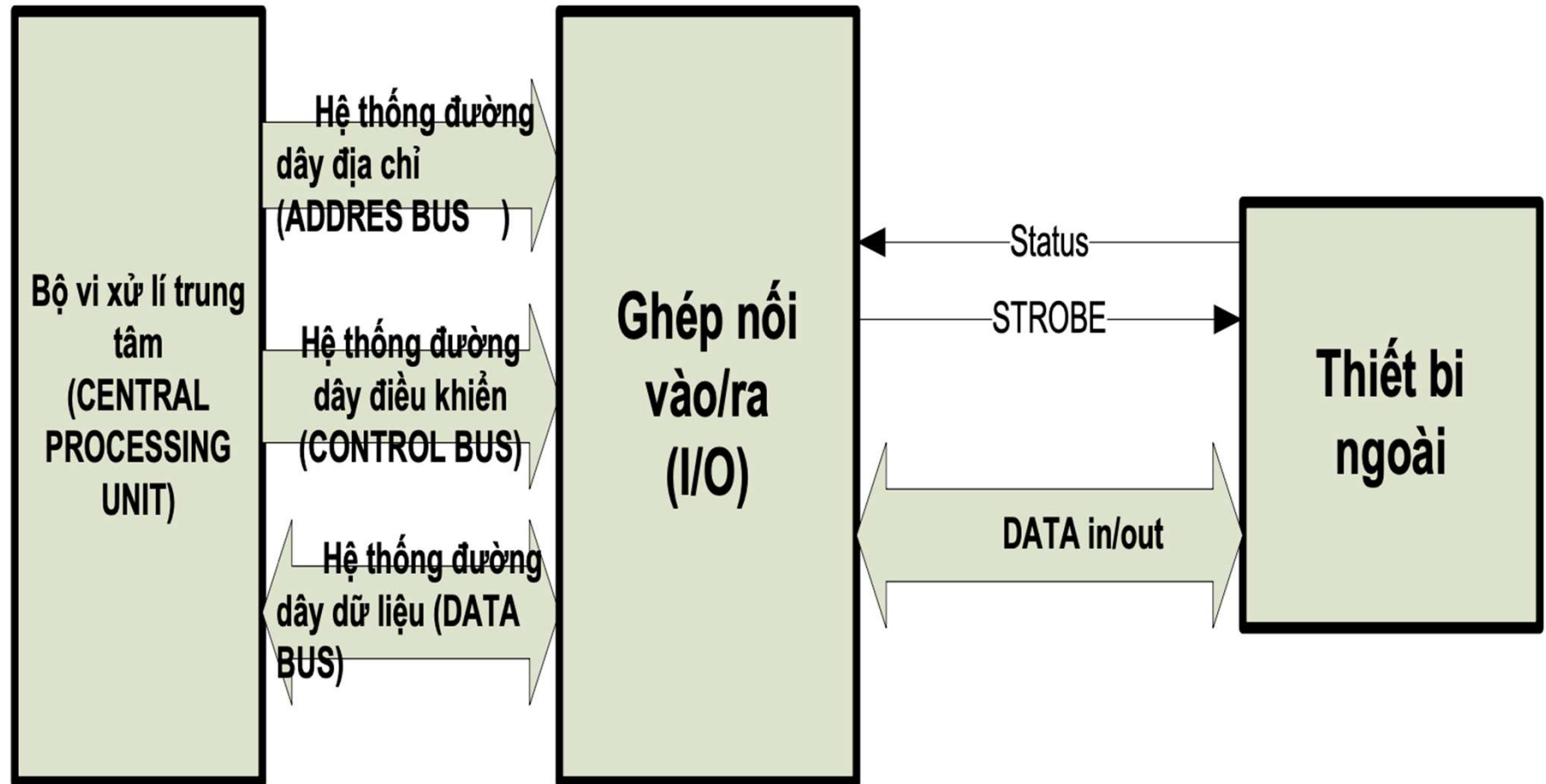
- 8 chips are needed
- 4164 DRAM chips have a capacity of 64 Kbit/chip and are organized in a matrix (256 rows x 256 columns).
- To select a bit in the matrix, the address at the row and column position is determined by two decoders.
 - Each decoder needs 8 bits as input to produce 256 outputs: A7-A0 for rows and A15-A8 for columns.
 - The CPU sends addresses to the SN 74257 multiplexer: A7-A0 latches into the DRAM using RAS pulses; A15-A8 using CAS/ pulses.
 - When designing the control logic circuit to generate RAS, CAS, and WE, the control signals generated by the CPU must be used..

Memory design

- ❖ Dynamic RAM (DRAM) Design: MK 4164 DRAM is a 64K-bit DRAM on a single chip. The assumption is to design RAM for an 8085 CPU with a maximum RAM of 64 KB.



❖ Model of coupling technique



- ❖ A typical interconnection model in an embedded system usually includes three core components:
 - BUS System
 - Address BUS: Determines which peripheral is being accessed by the CPU.
 - Data BUS: Transmits data bidirectionally between the CPU and the peripheral.
 - Control BUS: Carries signals such as Read, Write, Enable, etc., to coordinate operations.
 - Interface/Ports: An intermediate layer between the CPU's BUS and the peripheral device.
 - Latch: Maintains data or address stability.
 - 3-State Buffer: Allows multiple devices to share the BUS without conflict.
 - Address Decoder: Generates chip select/port select signals.
 - Peripheral Device: Provides data for the CPU to process, or receives control commands from the CPU to perform external actions.

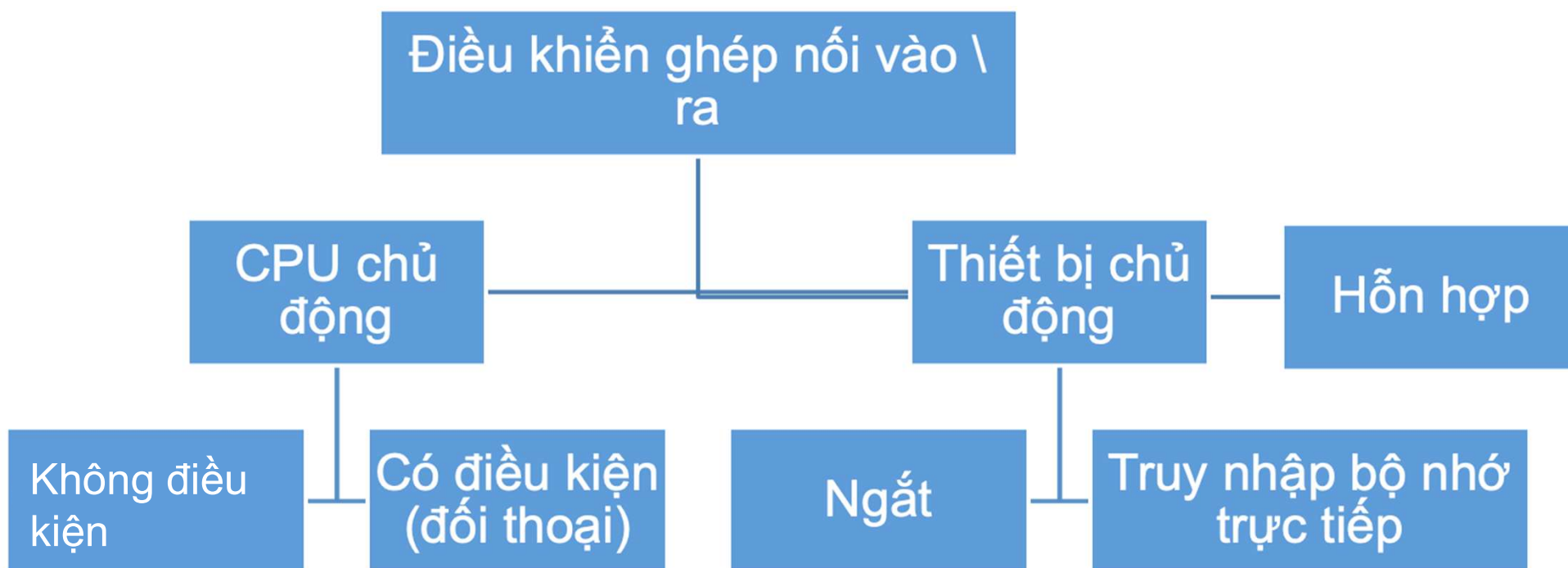
❖ Conventional CPUs have two methods of using addresses

- Using addresses to access memory, through memory-related machine instructions

Thiết bị	Phân bổ địa chỉ	Kích thước
RAM	0000 - 7FFF	RAM 32 KB
Cho các cổng sẽ dùng (I/O ports)	8000 - 80FF	256 bytes
Sound controller (I/O ports)	9000 - 90FF	256 bytes
Video controller/text-mapped display RAM	A000 - A7FF	2KB
ROM	C000 - FFFF	16 KB

- Using a very small number of special instructions to access devices (programmable integrated circuits) to send control codes to those devices.
 - The CPU accesses devices via ports.
 - For example, the instructions: IN des, src; OUT des, src
 - *src is usually DX containing [port_in_address] or [port_out_address]*
 - *des is usually AL or AX containing data to be read or written.*

❖ Connection types



❖ Connection types

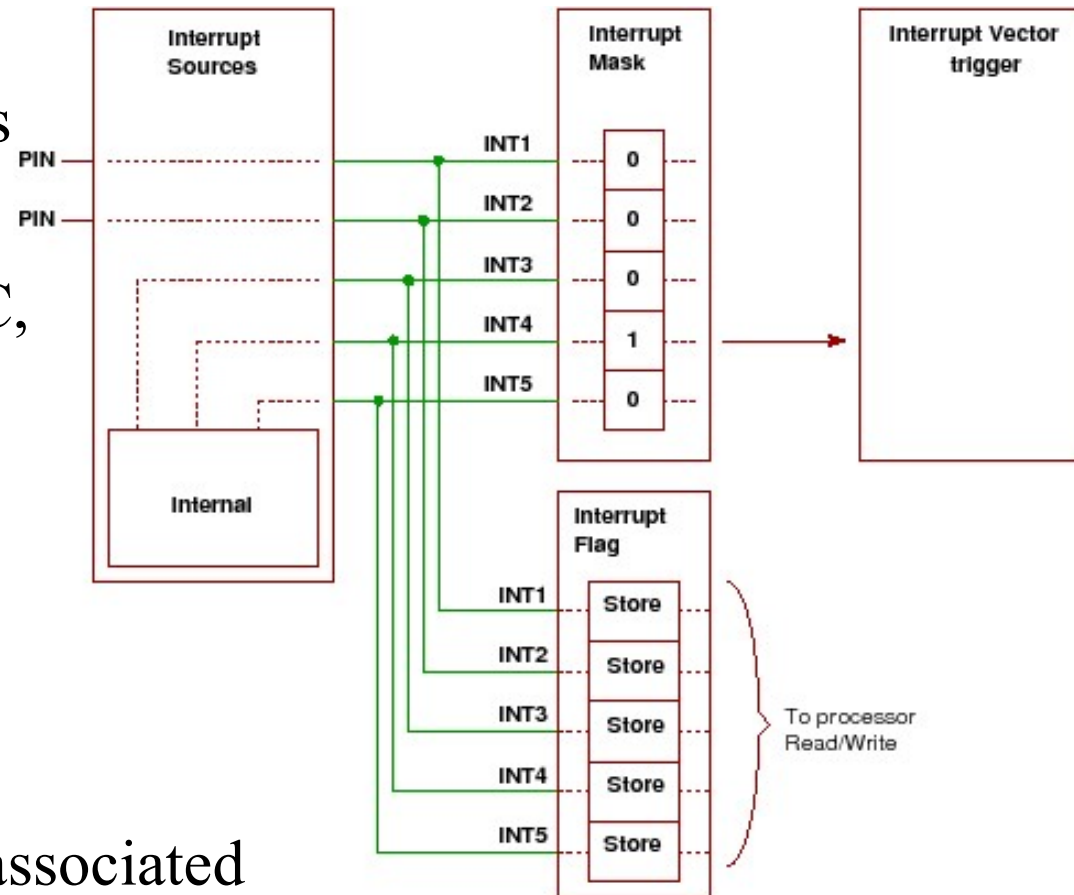
- CPU-active pairing: all operations are decided by the CPU.
 - Unconditional pairing: the CPU assumes the device is always ready, without checking its status. Suitable for very simple devices (LEDs, switches, relays)
 - Advantages: Simple circuit and software.
 - Disadvantages: Not secure with slow devices.
- Conditional pairing (Handshaking): The CPU must check the device's status before exchanging data through a status port.
 - If the device is not ready ($READY = 0$), CPU will: Wait or check again later.
 - This ensures accurate synchronization and avoids data loss.
- Polling: The CPU checks the status of multiple peripheral devices sequentially in a defined order.
 - If a device is ready $\rightarrow$ the CPU serves it immediately. Then it continues checking other devices
 - Easy to set up and control, but wastes processing time if few devices are active

❖ Connection types

- I/O-active coupling: The peripheral device is the active party; the CPU only responds when needed.
 - Interrupt
 - When a device needs servicing: It sends an IRQ signal to the CPU.
 - CPU: Pauses the main program, saves the state, jumps to the interrupt service routine (ISR), processes the interrupt, and returns to the previous task.
 - Advantages: The CPU doesn't have to wait, high CPU utilization efficiency, suitable for real-time systems. This is a very common method in microcontrollers and RTOS.
 - Direct Memory Access – DMA: The CPU does not directly transmit data. Instead, the DMA controller (DMAC) will: Gain control of the BUS, transmit data directly between RAM and the device.
 - During this time: The CPU temporarily "gives up the BUS", but can still process other tasks.
 - DMA helps: Increase data transmission speed, reduce CPU load.

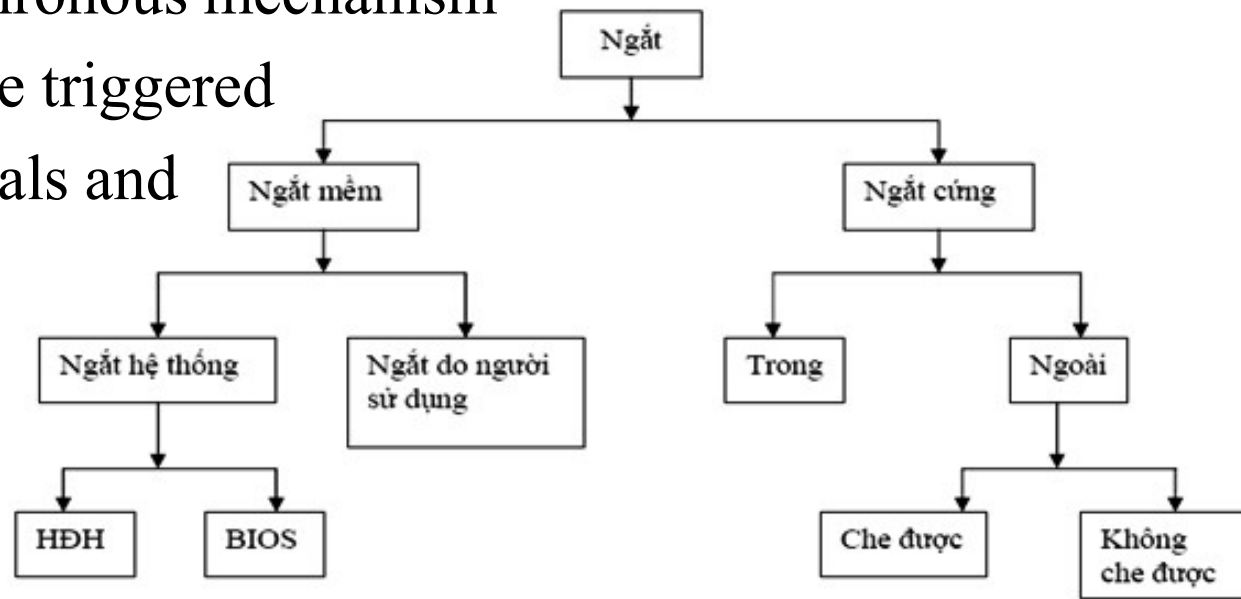
❖ A basic interrupt system:

- Interrupt sources: interrupt signals from devices on or outside the board, for example from ADC/DAC, from UART, from interface ports, DMAC.
- Interrupt flags: Each interrupt is associated with a flag, only 1 bit, indicating the state of that interrupt.
- Interrupt mask: Each interrupt is associated with 1 bit of the mask register. Programming the bits of the register will enable or disable the corresponding interrupt.



❖ Interrupt Types:

- Software interrupts essentially perform a special function call triggered by interrupt sources, which are events originating from within the program and on-chip peripherals. This is a program-synchronous interrupt mechanism.
- Hardware interrupts can be viewed as a special function call where the trigger source is an event coming from outside the program via a hardware structure. This is an asynchronous mechanism because interrupt events are triggered by external peripheral signals and are relatively independent of the CPU.



❖ Basic Interfacing Methods

- GPIO (Digital I/O)
 - Operating Method:
 - HIGH (1) $\rightarrow$ 3.3V or 5V,
 - LOW (0) $\rightarrow$ 0V
 - Applications: turning LEDs on/off, reading push buttons, controlling relays.
- Analog Interface
 - ADC (Analog $\rightarrow$ Digital)
Reads analog signals: temperature, voltage, light
 - DAC (Digital $\rightarrow$ Analog)
Outputs analog signals: audio, controlling analog devices
- PWM (Pulse Width Modulation)
 - PWM generates pulse signals with a variable duty cycle.
 - Applications: motor speed control, LED brightness adjustment, servo control

❖ Peripheral connection using communication bus:

- When the device is complex, a data transmission bus is needed.
- Basic connection standards: UART, SPI, I2C

	UART	SPI	I2C
Đồng bộ	✗	✓	✓
Số dây	2	4+	2
Tốc độ	TB	Rất cao	TB
Nhiều thiết bị	✗	✓	✓
Địa chỉ	✗	✗	✓

- Advanced connection standards: CAN, USB, Ethernet

❖ Peripheral Connections via Communication Bus:

- UART (Universal Asynchronous Receiver/Transmitter)
 - Characteristics:
 - Asynchronous serial communication,
 - No common clock required,
 - Configurable baud rate
 - Connection wires: TX (transmit), RX (receive), GND
 - Data frame: START | DATA | PARITY | STOP
 - Advantages: Simple, easy to debug
 - Disadvantages: Point-to-point only, average speed
 - Applications: Debugging serial monitors, GPS modules, Bluetooth modules
 - Choose UART when:
 - PC communication is needed,
 - system debugging,
 - simple configuration

❖ Peripheral Connections via Communication Bus :

■ SPI (Serial Peripheral Interface)

• Characteristics

- Synchronous communication
- Very high speed
- Full duplex

• Connection wires

Dây	Chức năng
MOSI	master → slave
MISO	slave → master
SCLK	clock
SS/CS	chọn thiết bị

- Advantages: Very fast, bidirectional transmission, stable
- Disadvantages: Multiple wires, each slave requires one SS pin
- Applications: TFT displays, SD cards, external flash memory, high-speed sensors
- Choose SPI when: High speed is required, large data transmission, displays, memory cards

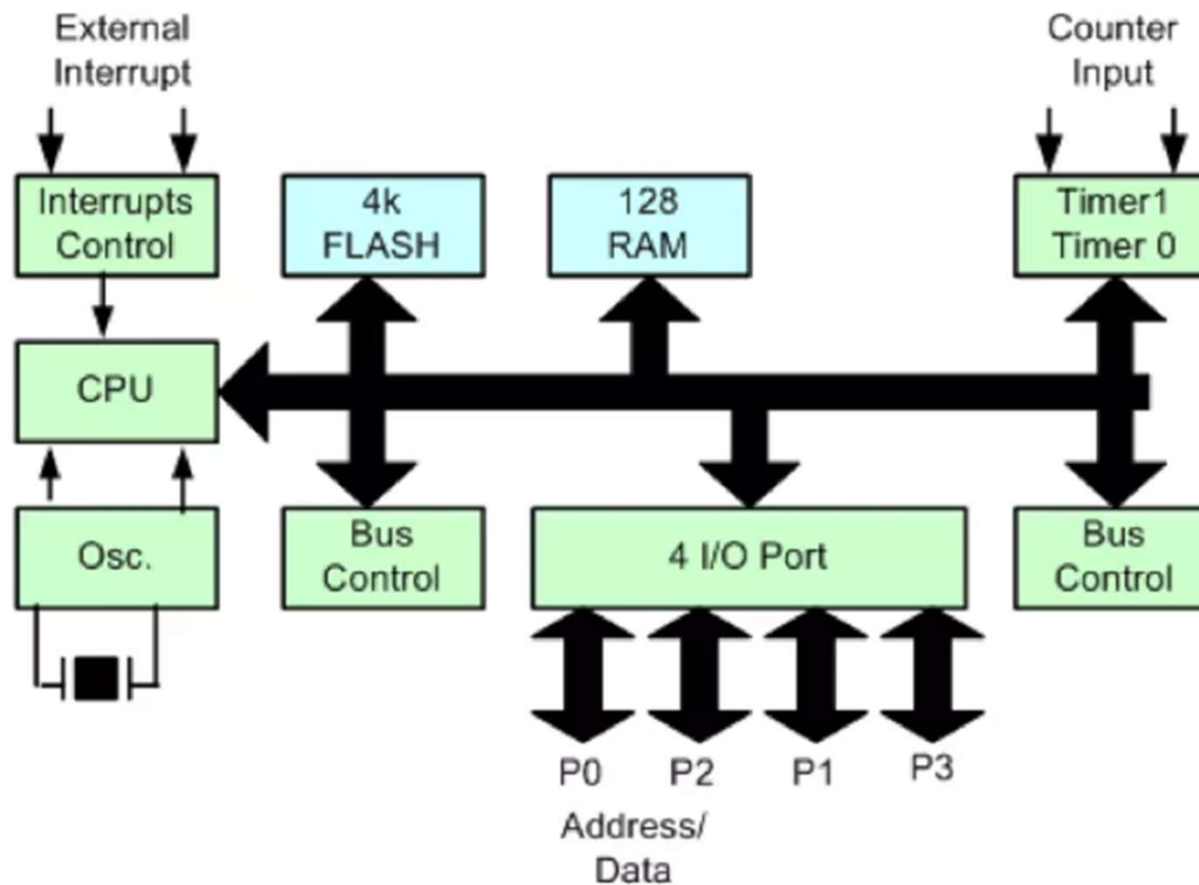
❖ Peripheral Connections via Communication Bus :

- I2C (Inter-Integrated Circuit)
 - Characteristics:
 - Multiple devices on the same bus,
 - each device has an address
 - Connection wires :
 - Only 2 wires: SDA (data), SCL (clock)
 - How it works: Master sends address → slave responds.
 - Advantages: saves wiring, supports multiple devices, easy to expand
 - Disadvantages: slower than SPI, short range
 - Applications: temperature sensors, OLED displays, RTCs, EEPROM
 - Choose I2C when :
 - multiple sensors,
 - need to minimize wiring,
 - low-speed device

❖ Peripheral Connections via Communication Bus :

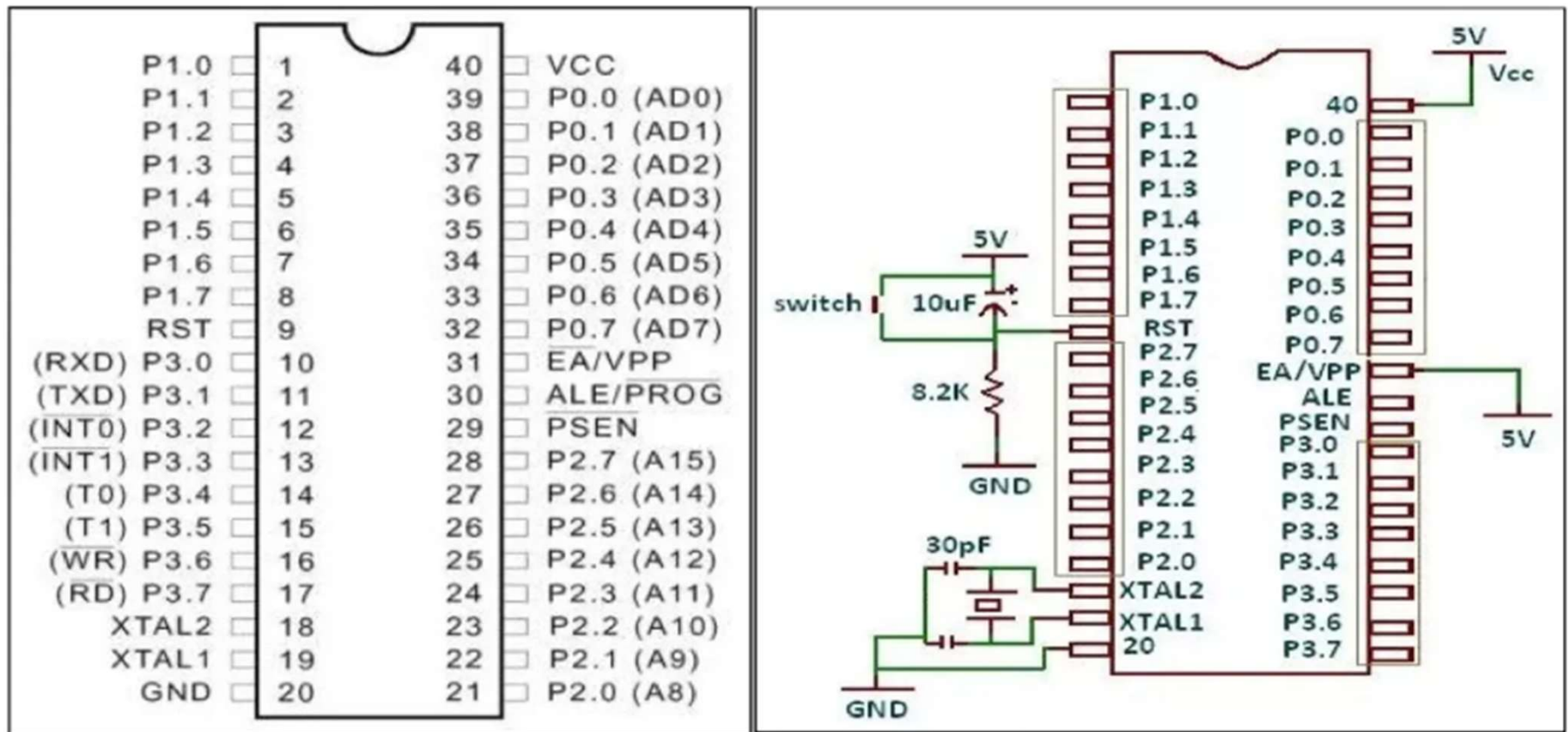
- Advanced connection standards (for reference)
 - CAN Bus
 - Used in automobiles and industry
 - Good noise immunity
 - USB
 - High speed
 - Connects peripheral devices
 - Ethernet: Network data transmission

❖ 8051 chip block diagram:



8051 microcontroller

❖ 8051 chip block diagram: There are 32 input/output pins; however, 24 of these 32 pins are multi-purpose.



❖ 8051 chip block diagram:

- Port 0 (pins 32-39), labeled P0.0-P0.7, has two functions.
 - Port 0 is used for input/output
 - In larger designs with external memory, Port 0 becomes a multiplexed address bus and data bus (low address byte).
- Port 1 (pins 1-8), labeled P1.0-P1.7, is solely for input/output and is used for communication with external devices.
- Port 2 (pins 21-28), labeled P2.0-P2.7, has two functions
 - Input/Output Operations
 - the high address byte of the 16-bit address bus for designs with external program memory or designs with more than 256 bytes of data memory.
- Port 3 (pins 10-17), labeled P3.0-P3.7, has two functions.
 - Input/Output Operations
 - The pins of port 3 have various functions.

❖ 8051 chip block diagram:

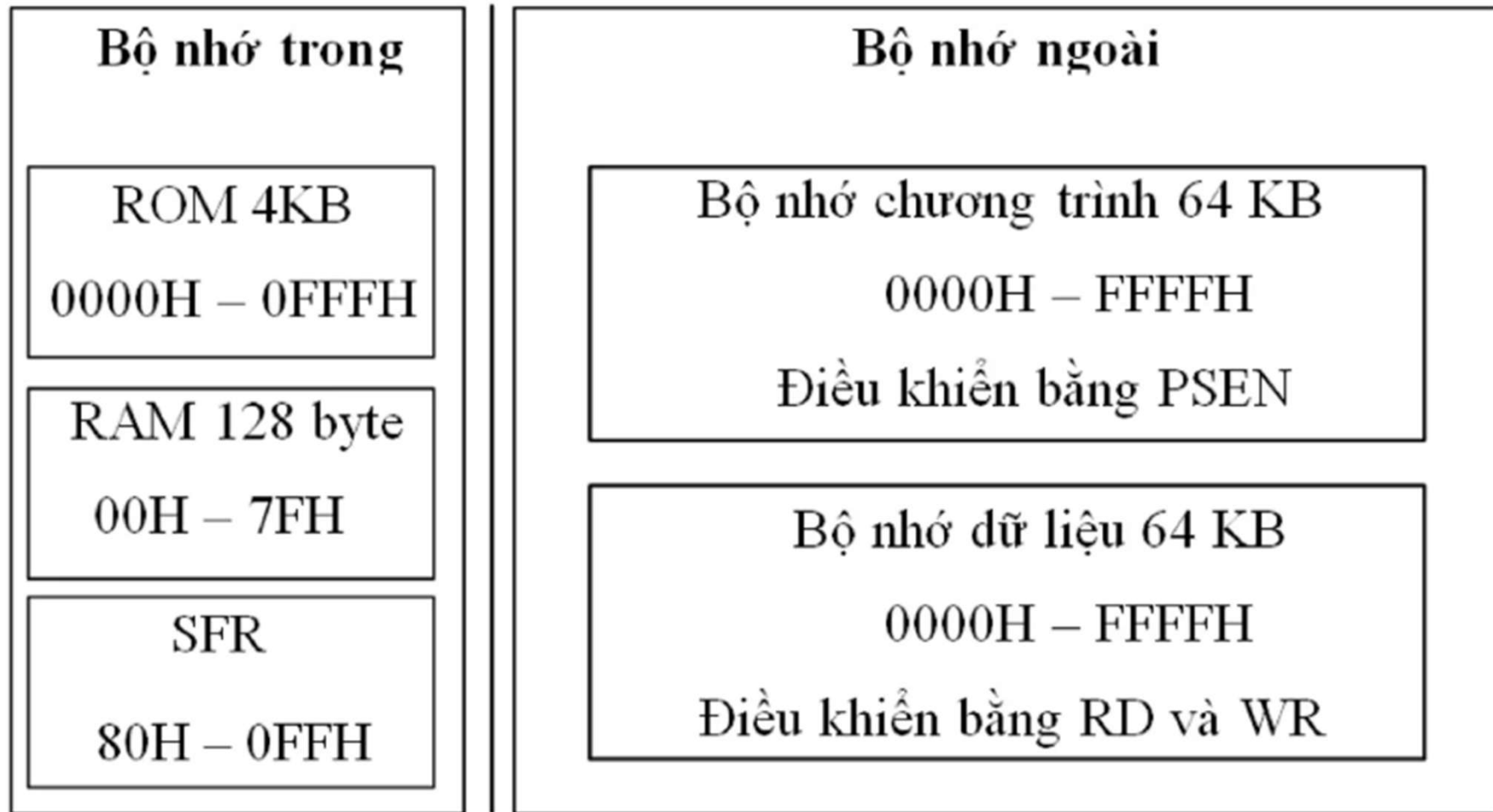
- PSEN: Pin 29
 - Enables access to external program memory. The /PSEN signal is at logic 0 during instruction fetching.
 - When executing a program stored in internal ROM, the /PSEN pin is maintained at a non-active logic level (logic 1).
- ALE: Used for demultiplexing the data bus and address bus.
- EA:
 - If this pin is connected to 5V, the 8051 chip executes the program in internal ROM.
 - If this pin is connected to GND (and the /PSEN pin is also at logic 0), the program to be executed is stored in external memory.
- The on-chip oscillator is coupled to an external crystal oscillator at pins XTAL1 and XTAL2.

❖ 8051 chip block diagram:

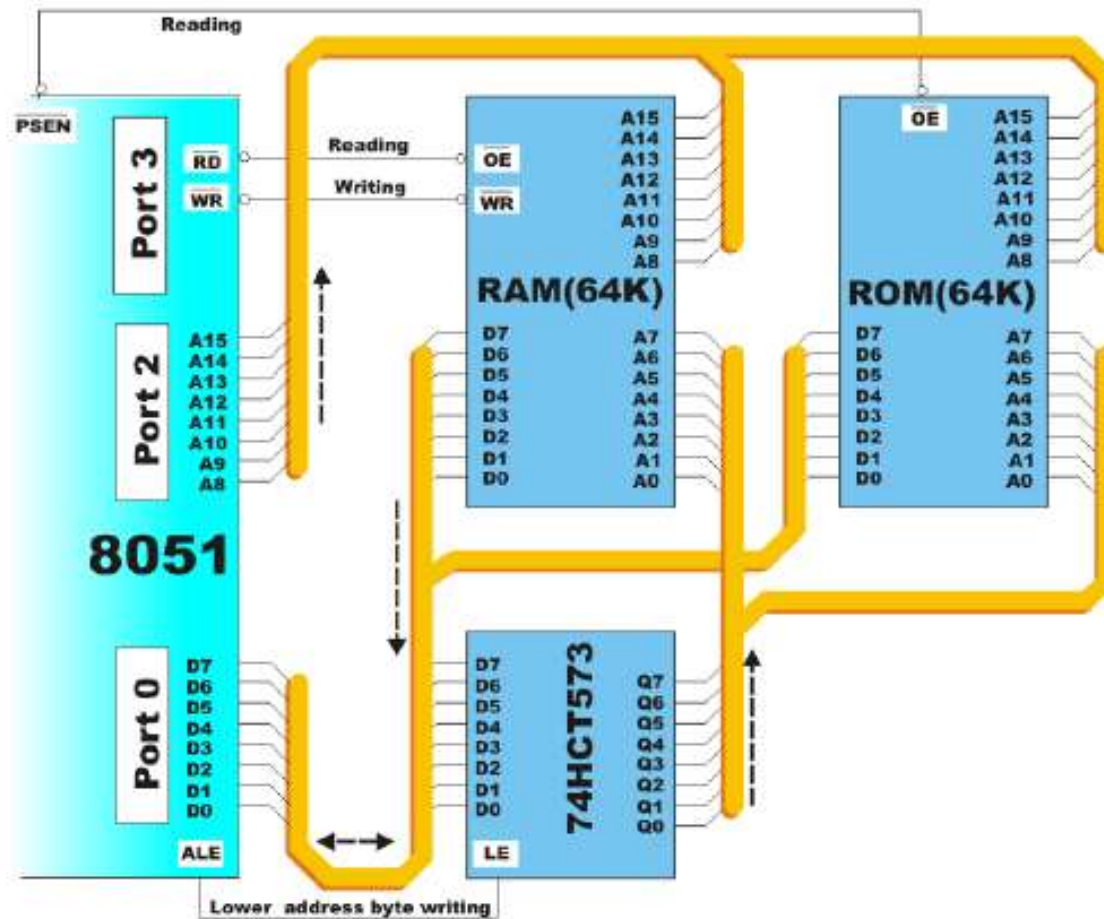
- RESET (RST): When this input is held at logic level 1 for at least 2 machine cycles, the 8051's internal registers are reloaded with the appropriate values for system restart.

Thanh ghi	Nội dung
Bộ đếm chương trình	0000H
Thanh chứa A	00H
Thanh ghi B	00H
PSW	00H
SP	07H
DPTR	0000H
Port 0-3	FFH
IP	xxx00000B
IE	0xx00000B
Các thanh ghi định thời	00H
SCON	00H
SBUF	00H
PCON (HMOS)	0xxxxxxxB
PCON (CMOS)	0xxx0000B

❖ Memory organization of the 8051 :

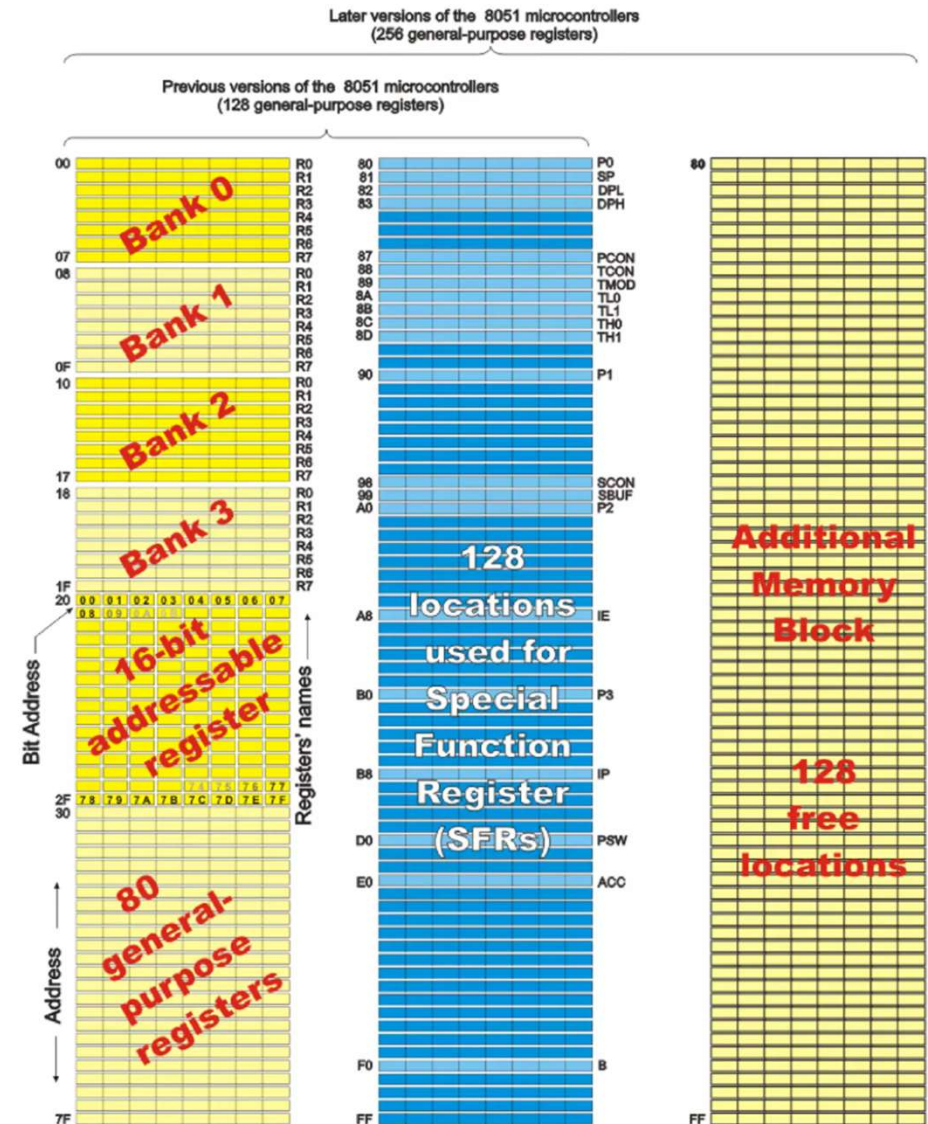


❖ Example of memory expansion design :

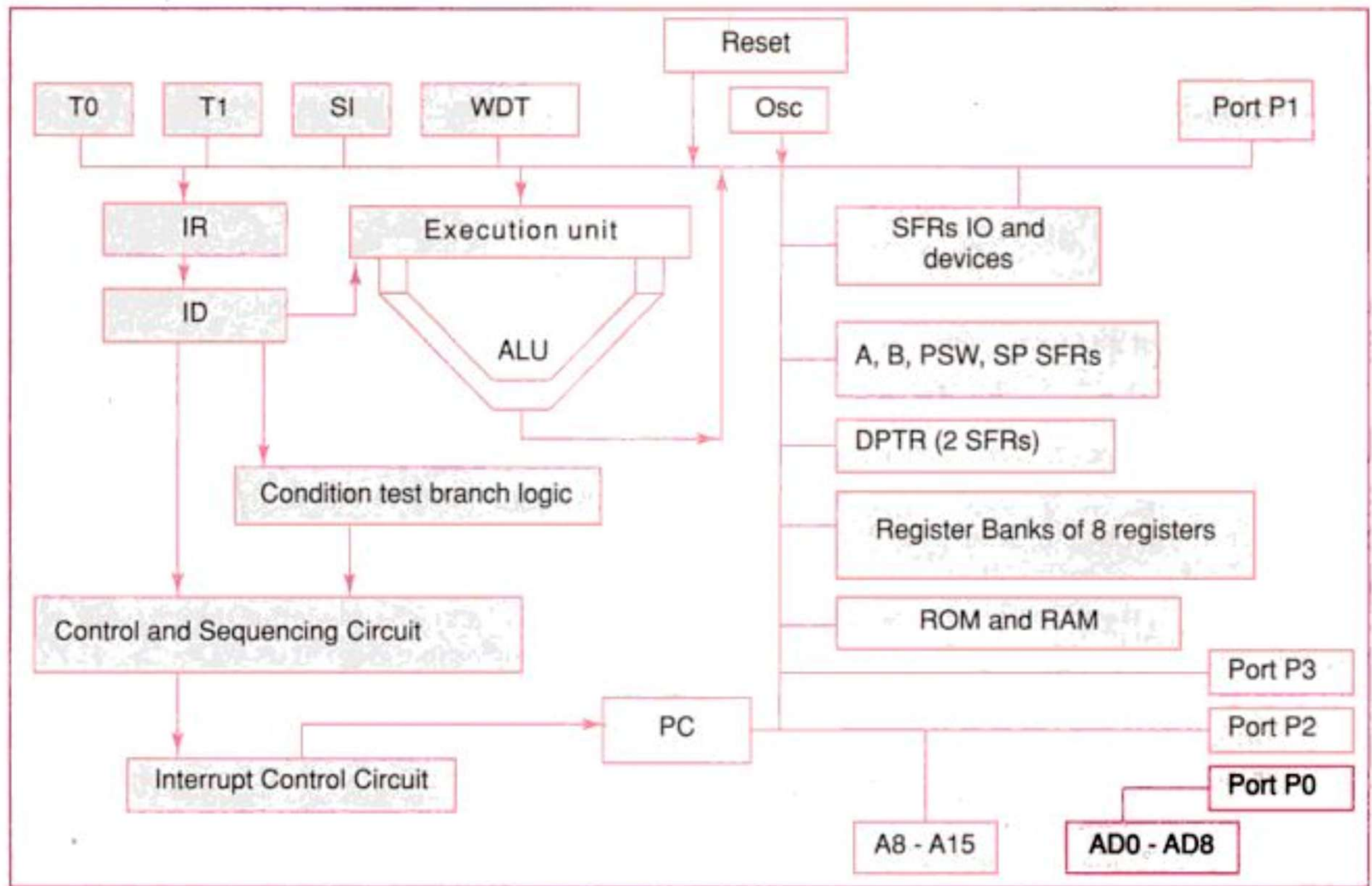


❖ RAM and Special Registers Distribution Diagram:

- 21 Special Function Registers (SFRs) occupy the upper part of the internal RAM from address 80H-FFH.
- Acc, B registers
- Stack Pointer (SP)
- Data Pointer (DPTR): used to access
- external memory
- Port Registers
- Timer Registers
- Serial Port Registers
- Interrupt Registers



❖ 8051 Basic Architecture



- ❖ Instruction Set Architecture
- ❖ Addressing Modes8051
- ❖ Input/Output Programming
- ❖ Interfacing with External Memory
- ❖ Counters and Timers
- ❖ Serial Data Input/Output8051
- ❖ Interrupts
- ❖ System Bus Communication

8051 microcontroller

KIT HỌC TẬP 8051 BẢN PRO

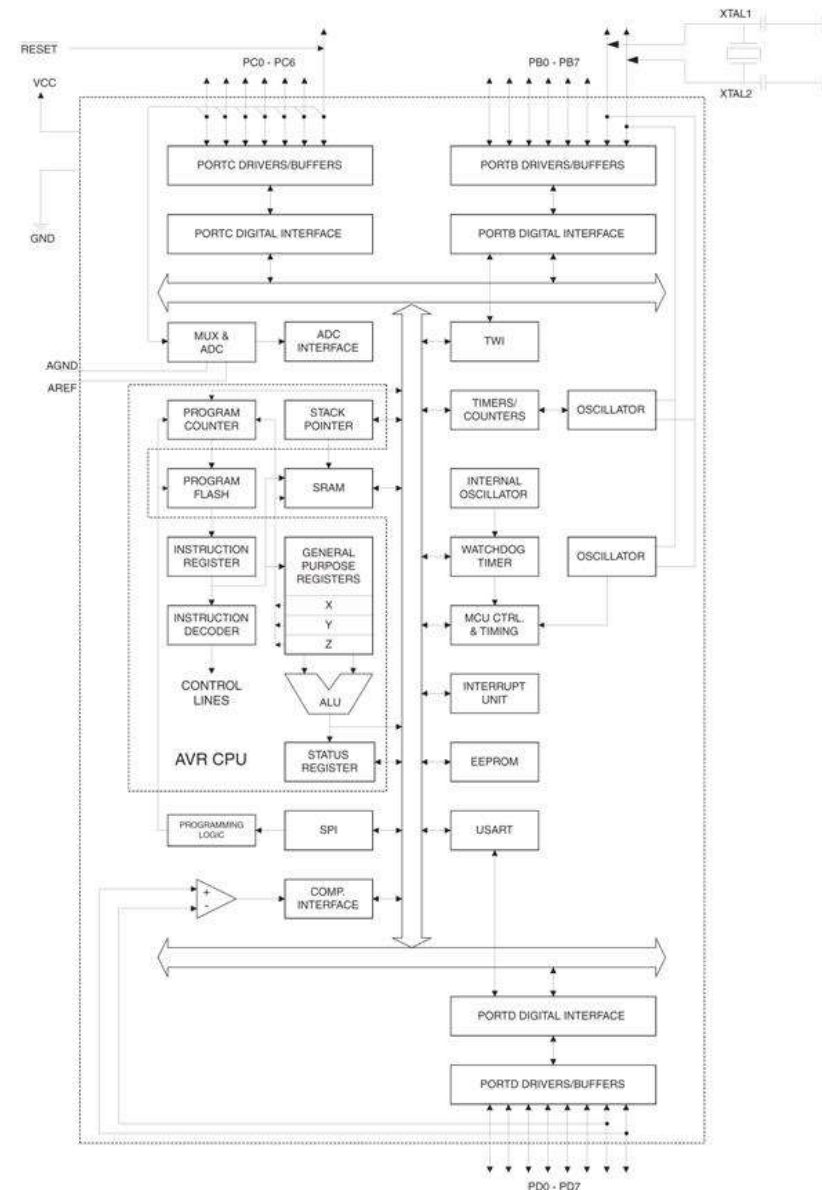
Bộ sản phẩm gồm:

- + IC 89C52RC
- + Dây usb kết nối
- + Motor DC
- + Điều khiển Remote 21
- + LCD
- + Cánh quạt
- Gói phụ kiện không bao gồm mạch nạp



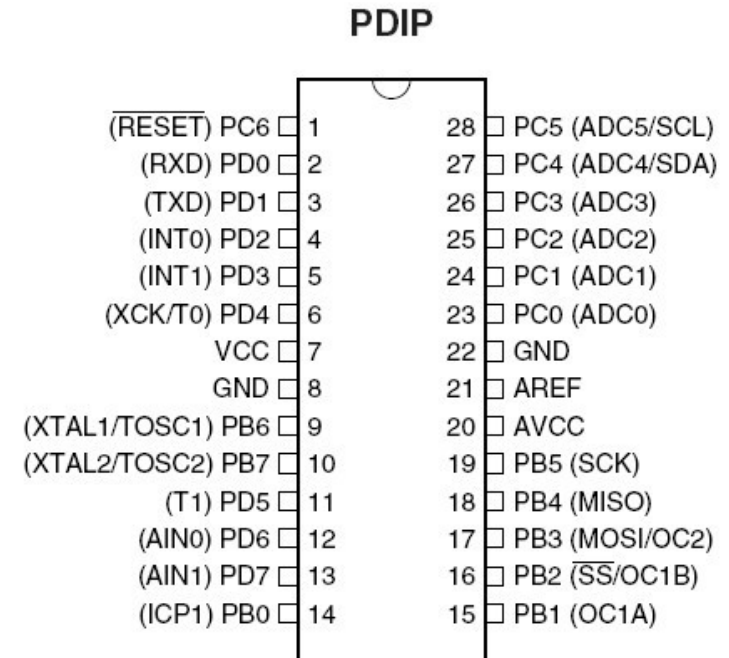
Atmega8 microcontroller

- ❖ The ATMEL Atmega8 microcontroller is a new type of AVR microcontroller.
 - The Atmega8, with its RISC architecture, boasts high performance and low power consumption.
 - It features 8KB of Flash memory, 512 bytes of EEPROM memory, and 1KB of SRAM memory.
 - Two 8-bit counters/timers with separate comparator and frequency divider modes.
 - One 16-bit timer with comparator mode.
 - A real-time clock (RTC) with a separate oscillator.
 - An 8-bit PWM pulse width modulator.
 - An internal 8-channel 10-bit ADC converter.
 - Two programmable serial USARTs.
 - A programmable watchdog timer with an on-chip oscillator.



❖ Pinout Diagram:

- VCC: Power supply voltage.
- GND: Ground.
- Port B (PB0...PB7), Port C (PC0...PC7), Port D (PD0...PD7) are 8-bit I/O ports.
- Port C also acts as the 8 high address lines from A8 to A15 when connecting external SRAM memory.
- Reset: Input is reset.
- AVCC: Provides power to Port C and the ADC converter.
- AREF: This is the reference voltage adjustment pin for the A/D converter.
- XTAL1: Input to the inverting amplifier and external clock signal generator.
- XTAL2: Output to the inverting amplifier.



Vi điều khiển Atmega8

- ❖ The Arduino UNO can use three 8-bit AVR microcontrollers: ATmega8, ATmega168, and ATmega328.

